Physics of Fluids

RESEARCH ARTICLE | MAY 04 2026

# Multi-turbine turbulent wake simulation using physics-informed neural networks

Azhar Gafoor CTP ; Sumanth Kumar Boya ; Rishi Jinka ; Abhineet Gupta ; Deepak N Subramani

Check for updates

*Physics of Fluids* 38, 055108 (2026)
https://doi.org/10.1063/5.0322331

View Online Export Citation

Articles You May Be Interested In

A physics-informed neural network for turbulent wake simulations behind wind turbines

*Physics of Fluids* (January 2025)

Characterization and classification of offshore wind profiles via a shape-based cluster analysis framework

*Physics of Fluids* (July 2025)

12 September 2026 13:03:32

AIP Publishing

Physics of Fluids

Special Topics Open for Submissions

Learn More

AIP Publishing

Physics of Fluids ARTICLE pubs.aip.org/aip/pof

# Multi-turbine turbulent wake simulation using physics-informed neural networks

Cite as: Phys. Fluids **38**, 055108 (2026); doi: 10.1063/5.0322331
Submitted: 12 January 2026 · Accepted: 6 April 2026 ·
Published Online: 4 May 2026

View Online Export Citation CrossMark

Azhar Gafoor CTP,[1,a)] Sumanth Kumar Boya,[1] Rishi Jinka,[1] Abhineet Gupta,[2] and Deepak N Subramani[1]

## AFFILIATIONS

[1]Department of Computational and Data Sciences, Indian Institute of Science, Bangalore, Karnataka 560012, India
[2]Shell India Markets Pvt Ltd., Bangalore, Karnataka 560103, India

[a)]Author to whom correspondence should be addressed: azharctp@iisc.ac.in

## ABSTRACT

In real-world fluid dynamics, turbulence is prevalent, from swirling coffee in a cup to galaxy formation. Physics-informed neural networks (PINNs) are pivotal in scientific machine learning and have been applied to engineering turbulence problems; however, they encounter challenges due to the chaotic and computationally intense nature of turbulence simulations. We introduce a PINN model for simulating turbulent wakes in multi-turbine configurations using a hybrid physics-data training strategy. Our method integrates data losses with boundary and partial differential equation losses, as formulated by the 2-equation $k-\varepsilon$ turbulence model. Using sparse data for a single flow variable restricted to a plane, the model simulated flow variables in the three-dimensional domain, showing excellent agreement with the reference (shear stress transport k–$\omega$ numerical solver) in various scenarios. Training data are viewed as simulated observational data on a coarse grid, 50× coarser than what the numerical solver requires. A transfer learning strategy is implemented to accelerate training in new cases, achieving 7.5× faster speed with only one-fourth of the data compared to training from scratch. Our results indicate that the hybrid training method for PINNs can be successfully applied in practical scenarios with limited sensory data to create neural surrogates for multi-turbine wind farms.

*Published under an exclusive license by AIP Publishing.* https://doi.org/10.1063/5.0322331

12 September 2026 13:03:32

## I. INTRODUCTION

As wind energy becomes increasingly crucial for sustainable power generation, optimizing wind farm layouts and turbine performance requires accurate prediction of turbulent wake interactions between multiple turbines. The complex wake structures that form behind wind turbines significantly affect downstream power generation, structural loads, and overall farm efficiency. Understanding and predicting these wake interactions is fundamental to advance wind energy technology and maximize renewable energy output.

The underlying physics of turbulent flows, governed by the Navier–Stokes (NS) equations, represents a challenging problem in physics with far-reaching implications for weather forecasting, aircraft design, ocean current prediction, and renewable energy systems. These partial differential equations form the foundation of modern fluid dynamics. The chaotic nature of turbulence, combined with the non-linearity of the governing equations, presents formidable challenges for developing accurate and efficient simulation methodologies.

Traditionally, the Navier–Stokes equations have been tackled numerically using direct approaches such as Direct Numerical Simulation (DNS),[1] which explicitly resolves all turbulent temporal and spatial scales but becomes computationally infeasible at high Reynolds numbers. To obtain more practical formulations, turbulence closure strategies are employed. Large Eddy Simulation (LES)[2] explicitly resolves the large, energy-containing eddies and represents the remaining unresolved subgrid-scale (SGS) motions through an SGS closure model; the near-wall region in LES can be treated as wall-resolved or wall-modeled, contingent on the Reynolds number and the available grid resolution. The Reynolds-Averaged Navier–Stokes (RANS) equations[3] further decrease computational demands by solving for time-averaged (or ensemble-averaged) flow fields, with all turbulent fluctuations accounted for through closure models. RANS thus constitutes a practical balance between computational expense and predictive accuracy, and it is therefore the dominant methodology in complex industrial simulations.[4–6] These flow problems are typically solved over discretized domains using finite difference, finite volume, finite element, or spectral methods.

Nevertheless, conventional numerical approaches face significant limitations. DNS and LES require very fine spatial discretization to


Published under an exclusive license by AIP Publishing

resolve the full range of turbulent length scales and very small time steps to capture temporal dynamics, which leads to extremely large computational meshes and prohibitive costs at high Reynolds numbers. Even RANS models, though less expensive, still necessitate high near-wall grid resolution and iterative solution of coupled nonlinear PDEs, which becomes expensive for complex geometries. Recent advances in neural networks and massively parallel training on Graphics Processing Units (GPUs) have created new opportunities in scientific machine learning (SciML), facilitating the construction of neural surrogate models for fluid simulation applications. GPU acceleration has also been incorporated into traditional CFD solvers. Notable examples are AFiD[7] and CaNS[8] for incompressible flows, STREAmS[9] and URANOS[10] for compressible turbulent flows, and FluTAS[11] for multiphase simulations. While these GPU-accelerated CFD solvers substantially decrease wall-clock time, several core limitations remain. Communication overhead across GPU nodes is still a major bottleneck, and GPU-accelerated solvers continue to share the inherent restrictions of conventional CFD: they depend on structured or unstructured mesh generation for each new geometry, and these approaches must also re-solve the governing equations from the beginning whenever the flow configuration, boundary conditions, or geometric parameters change, requiring full re-initialization and iterative convergence.[12] This makes systematic parametric studies (e.g., examining the impact of varying inlet conditions, turbulence intensities, or geometric parameters across many configurations) and real-time deployment computationally infeasible.[13,14]

A widely used class of SciML frameworks for solving partial differential equations (PDEs) is operator-learning methods, including DeepONet,[15] the Fourier Neural Operator,[16] and Representation Equivalent Neural Operators.[17] These methods, which learn mappings between function spaces, have demonstrated improved generalization in PDE solution tasks by employing novel architectures that map spatial coordinates, geometry, and operating conditions to flow field variables. Nevertheless, the operator-learning methods often demand large amounts of high-quality data for training, with their performance significantly declining in data-scarce situations, due to which these approaches often face limitations when applied to high-dimensional turbulent flows, particularly for three-dimensional and multi-source-term PDEs.[18] While the operator-learning methods have seen limited adoption specifically for wake modeling and wind power forecasting applications, numerous data-driven neural network models have been proposed for these tasks. Examples include models based on Artificial Neural Networks (ANN)[19–21] or a combination of Proper Orthogonal Decomposition (POD) with Long Short-Term Memory networks,[22] and convolutional neural networks (CNNs).[23] These data-driven approaches rely heavily on large, high-quality training datasets, which are often difficult to obtain in real-world settings, or generating such simulation data are resource-intensive, requiring significant computational power and time, often running traditional numerical methods on multiple-core systems for extended periods[24] which limits the broader adoption of these methods in engineering applications.

Physics-Informed Neural Networks (PINNs) provide a promising alternative through a loss function based on PDE-residuals for semi-supervised or unsupervised training.[25] In the standard PINN formulation, the total loss comprises multiple terms: the PDE residual evaluated at the collocation points, boundary condition (BC) residuals enforced at domain boundaries, and for time-dependent problems, initial condition (IC) residuals enforced at the initial time. By integrating PDE residuals with standard data-driven loss components, PINNs alleviate the need for extensive simulation data. Moreover, they employ automatic differentiation[26] for computing PDE residuals, removing the requirement for complex mesh generation typical of traditional solvers. The ability of neural networks to assimilate various fidelity measurements alongside physical constraints makes PINNs particularly useful when high-quality data are scarce, but some experimental measurements are available.[14] Despite these strengths, PINNs face well-documented challenges when applied to stiff, multi-scale fluid systems such as high-Reynolds-number turbulence, including gradient pathologies and optimization instability.[27,28]

Raissi *et al.*[25] initially demonstrated the capability of PINNs to simulate fluid dynamics with minimal data for standard test cases. Advancements followed with two-dimensional laminar flow solvers for airfoil shapes,[29] and Navier–Stokes Flow nets,[30] a turbulent flow solver for the Navier–Stokes equations. Promising outcomes have been achieved in turbulent scenarios, including inverse solvers with PINN models adopting RANS formulations,[31] and NVIDIA SimNet's use in conjugate heat transfer problems.[32] Recent studies have presented PINN-based surrogate models for turbulent flow[33] and RANS-PINN schemes for estimating unknown turbulence equation components.[34] In the three-dimensional setting, Biswas and Anand[35] employed PINNs to model laminar flows governed by the Navier–Stokes equations, demonstrating that PINNs can address complex 3D fluid dynamics without conventional numerical schemes. More recently, Hu *et al.*[36] proposed a hybrid strategy that combines a physics-informed neural network with a characteristic-based split algorithm, reducing labeled data requirements, memory consumption, and computational time, while enhancing accuracy and convergence for 3D incompressible Navier–Stokes flows. PINNs have likewise been applied to reconstruction problems, including the reconstruction of the 3D wind field around a building model in a wind tunnel using a PINN framework incorporating a zero-equation turbulence closure,[37] the reconstruction of turbulent velocity and pressure fields from under-resolved, noisy particle trajectory measurements,[38] and the development of a physics-informed variational autoencoder for reconstructing 3D turbulent wall-bounded flow fields from sparse planar measurements.[39] The results show that the PINN framework provides an effective approach for simulating and reconstructing flow fields in engineering applications. PINNs have also been applied to various prediction tasks in wind farm flows. Specifically, NS equations in combination with sensory data are used in a PINN framework to estimate freestream wind fields in front of wind turbines.[40,41] Another study proposes a data-independent PINN surrogate to simulate three-dimensional wind turbine wakes based on the k–$\varepsilon$ turbulence model.[42] Different PINN variants are compared by Cobelli *et al.*[43] for reconstructing two-dimensional velocity field for a wind turbine with sparse data and NS equations. For wake field predictions, a PINN framework integrating two-dimensional NS equations, sensory data and actuator disk model is proposed to simulate the entire wake field.[44] Wang *et al.*[45] studied the dynamic nature of wake using the PINN framework with a moving average window technique. An improved PINN model with a cosine annealing algorithm and dynamic weight strategy, together with two-dimensional NS equations, sparse data, and the actuator disk model, is proposed for dynamic wake field prediction.[46] In general, the application of

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

PINNs in engineering flows is still in a preliminary stage, typically limited to laminar flows,[47,48] or benchmark examples in turbulent flows or turbulent flows in a two-dimensional regime.[49–51] We acknowledge that the modern literature on PINNs is extensive. As such, we restrict our focus to models for turbulent wake simulation, excluding other significant works in related sectors.

The current state-of-the-art RANS-based PINN model demonstrates excellent performance with single turbines.[42] Expanding PINN methods to study interactions between multiple wind turbine wakes is a crucial advancement. Simulating multiple turbine wakes involves complex challenges, such as wake interactions, diverse atmospheric conditions, and the need to model wake recovery and meandering that influence downstream turbines. These challenges remain largely unaddressed in the current literature on PINNs.[52–54] Furthermore, key questions remain on whether complex turbulence can be simulated without observational or simulation guidance during training.

In this work, we implement a hybrid training strategy within the PINN framework to simulate turbulent wake interactions across a grid of wind turbines. Our approach integrates simulated sparse pseudo-measurement data obtained from a RANS simulation using a traditional computational fluid dynamics (CFD) solver. The main contributions of this work are:

(1) Development of a PINN model to solve RANS equations for simulating turbulent flow in multi-turbine wind farm configurations using a hybrid physics-data training strategy.
(2) Comprehensive documentation of the challenges and solutions for training RANS simulations with PDE-residual loss terms in complex multi-turbine domains.
(3) Open-source release of code, model architecture, optimized hyperparameters, and trained models to enable direct application to multi-turbine wind farm simulations.
(4) Application of transfer learning to accelerate PINN training for turbulent wind turbine wake flows under varying atmospheric conditions.

In the remainder of the paper, we first discuss the development of the PINN model for simulating wake turbulence in multi-turbine arrays (Sec. II). After that, the results of the model development and simulation results are presented, including the calculation of the power output (Sec. III). The issues faced while training, computational time, and transfer learning strategy to reduce time are presented in the discussion (Sec. IV). Finally, a summary with future directions concludes the paper (Sec. V).

## II. DEVELOPMENT OF THE PHYSICS INFORMED NEURAL NETWORK

### A. Reynolds averaged Navier–Stokes equations with an actuator disk wind turbine model

For the multi-turbine turbulent wake simulation, we choose the Reynolds Averaged Navier–Stokes (RANS) equations with an actuator disk model for the wind turbine. The RANS equations are time-averaged equations designed to simulate turbulent fluid flows. They are formulated by decomposing turbulent flow variables into time-averaged and fluctuating components, followed by time averaging, resulting in a Reynolds-stress term $\tau_{ij}$. This stress term is modeled using the $k-\varepsilon$ turbulence closure model, which provides transport equations for turbulent kinetic energy ($k$) and dissipation rate ($\varepsilon$).[55]

The equations for incompressible steady RANS with $k$ and $\varepsilon$ transport equations in Cartesian coordinates are expressed in the form of implicit partial differential equations as follows

$$Continuity: f_c(\mathbf{u},\mathbf{x}) = \frac{\partial u_i}{\partial x_i} = 0, \tag{1a}$$

$$\text{Momentum: } f_m(\mathbf{u},\mathbf{x}) = \frac{\partial u_i u_j}{\partial x_j} + \frac{1}{\rho}\frac{\partial p}{\partial x_i} - \frac{\partial}{\partial x_j}\left[\nu\left(\frac{\partial u_i}{\partial x_j}+\frac{\partial u_j}{\partial x_i}\right)\right] + \frac{\partial \tau_{ij}}{\partial x_j} - S_i = 0, \tag{1b}$$

$$k-\text{transport: } f_k(\mathbf{u},\mathbf{x}) = \frac{\partial k u_j}{\partial x_j} - \frac{\partial}{\partial x_j}\left[\left(\nu+\frac{\nu_t}{\sigma_k}\right)\frac{\partial k}{\partial x_j}\right] - P_k + \varepsilon = 0, \tag{1c}$$

$$\varepsilon-\text{transport: } f_\varepsilon(\mathbf{u},\mathbf{x}) = \frac{\partial \varepsilon u_j}{\partial x_j} - \frac{\partial}{\partial x_j}\left[\left(\nu+\frac{\nu_t}{\sigma_\varepsilon}\right)\frac{\partial \varepsilon}{\partial x_j}\right] - C_{1\varepsilon}\frac{\varepsilon}{k}P_k + C_{2\varepsilon}\frac{\varepsilon^2}{k} - S_\varepsilon = 0, \tag{1d}$$

where $\mathbf{u} = (u, v, w, p, k, \varepsilon)$ represents the solution field and $\mathbf{x} = (x, y, z)$ denotes the spatial coordinates within the domain $\Omega$. Here, $u, v, w$ are the three-dimensional velocity components, $p$ is the mean pressure, $\nu$ is the kinematic viscosity, $\nu_t$ is the turbulent viscosity, $P_k$ is the production of turbulent kinetic energy (TKE), $S_i$ is the momentum source/sink term in the $i$ direction, and $S_\varepsilon$ is the source/sink term for dissipation rate.

The undefined terms in Eq. (1) are defined as follows:

$$\text{Reynolds}-\text{stress: } \tau_{ij} = -\nu_t\frac{1}{2}\left(\frac{\partial u_i}{\partial x_j}+\frac{\partial u_j}{\partial x_i}\right), \tag{2a}$$

$$\text{Turbulent viscosity: } \nu_t = C_\mu\frac{k^2}{\varepsilon}, \tag{2b}$$

$$\text{Prod. of TKE: } P_k = \nu_t\left(\frac{\partial u_i}{\partial x_j}+\frac{\partial u_j}{\partial x_i}\right)\frac{\partial u_i}{\partial x_j}. \tag{2c}$$

For the three-dimensional turbulent wake simulation around wind turbines under neutral atmospheric conditions, the constant values in Eqs. (1) and (2) are $C_\mu = 0.033$, $\sigma_k = 1.0$, $\sigma_\varepsilon = 1.3$, $C_{1\varepsilon} = 1.176$, and $C_{2\varepsilon} = 1.92$, determined by regression fits of observation data.[56]

The PINN framework employs gradient descent to optimize over a loss landscape comprising PDE, boundary, and data losses. Here, the loss landscape denotes the behavior of the objective function with respect to the network parameters, particularly its non-convex geometry, extended flat regions, and numerous local minima and saddle points. In contrast to conventional CFD solvers, which at each iteration resolve a well-posed linear system, a PINN jointly minimizes a composite loss that includes PDE residuals, boundary condition penalties, and data misfit terms. This multi-objective setup creates several optimization challenges: gradients from different loss components may conflict, driving parameter updates in opposing directions; large differences in the magnitudes of the loss terms can lead to severe ill-conditioning; and stiffness induced by multi-scale near-wall phenomena, where flow fields change rapidly over very short distances,

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

can generate extremely steep loss gradients that are difficult for standard gradient descent methods to navigate.[28,57–59] To address this, we incorporate wall functions directly into the loss function. In standard industrial flow applications, wall functions are employed to obtain accurate computational fluid dynamics simulations at a reduced computational expense.[60–62] These functions enforce a physically consistent near-wall velocity profile, thereby mitigating the scale separation between the wall and the freestream. By constraining the solution to adhere to the expected mean flow behavior in the region adjacent to the wall, the wall function acts as a regularizer for the optimization landscape and steers the model away from nonphysical local minima (see the Appendix D).

In the RANS governing equations, the wind turbine is represented by the actuator disk model, employing source terms $S_i$ and $S_\varepsilon$. The momentum source term ($S_i$) in Eq. (1b) applies thrust forces on the flow across the actuator disk's region. Thrust ($T$) and power ($P$) equations are

$$T = \frac{1}{2}\rho C_T A_d u_0^2, \tag{3}$$

$$P = \frac{1}{2}\rho C_P A_d u_0^3, \tag{4}$$

where $C_T$ is the thrust coefficient, $C_P$ is the power coefficient, $A_d$ is the disk area, and $u_0$ is the upstream velocity. $C_T$ is a dimensionless coefficient representing the axial force exerted by the turbine on the incoming flow, and $C_P$ is the ratio of the power extracted by the turbine to the wind energy. Manufacturers supply the thrust and power coefficients for modeling the turbine's components, such as blades and the generator. Although the actuator disk model lacks geometric details of the turbine, it effectively represents the mean wake impact and accurately calculates the power output in wind farm simulations.[63] In this study, the thrust force from Eq. (3) is averaged over the turbine area and distributed along the streamwise direction using the source term

$$S_i = -\frac{1}{2}\rho C_T u^2, \tag{5}$$

where $u$ represents the flow velocity measured at an upstream location equal to four turbine diameters, which is used as the reference for calculating the thrust coefficient (Sec. III A). The negative sign indicates that the thrust is applied to the incoming flow, making $S_i$ a momentum sink. The standard $k-\varepsilon$ RANS closure model, while predicting the disk's presence, often overestimates some flow variables. Thus, an additional term in the $\varepsilon$-transport equation [Eq. (1d)] is added to address this overestimation. This additional source term $S_\varepsilon$ captures the energy transfer rates from large to small-scale turbulence, influenced by the production range scale and the dissipation rate timescale,[64,65] and is expressed as follows:

$$S_\varepsilon = C_{4\varepsilon}\frac{P_k^2}{\rho k}, \tag{6}$$

where $C_{4\varepsilon}$ is set at 0.37 based on experimental data,[65] $P_k$ is the production of turbulent kinetic energy, and $k$ is the turbulent kinetic energy. In the current PINN formulation, the source terms are computed dynamically at every training iteration. Rather than prescribing the velocity in the source term as a fixed input, it is obtained from the PINN's own prediction of the velocity field at the specified upstream location relative to the turbine rotor. At each iteration, the network first infers this upstream velocity, which is then used to evaluate the source term and incorporate it into the residual of the momentum equation within the actuator disk region. This guarantees that the source term evolves in step with the predicted flow field over the course of training, enabling the network to converge to the solution in which turbine forcing and the surrounding velocity field are intrinsically linked.

### B. PINN model and loss function

In the physics-informed neural network method (PINN; Fig. 1), a parameterized neural model $\mathscr{H}$ is trained to approximate the

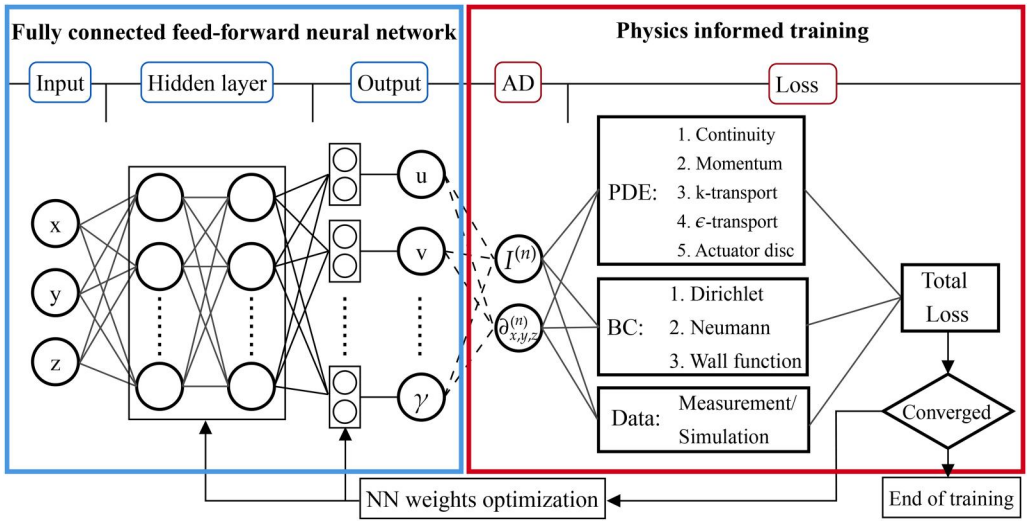



**FIG. 1.** *Schematic of a PINN model for three-dimensional RANS simulation.* The PINN model is a fully connected feed-forward dense neural network (NN) shown in the blue enclosure. The loss calculation through automatic differentiation (AD) is visualized as dotted connections in the red enclosure. Three losses are calculated (i) partial differential equation (PDE) residual loss, (ii) boundary condition (BC) loss, and (iii) data loss.

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

functional relationship between the input domain $\Omega$ and the solution domain $\mathscr{U}$, i.e., $\mathscr{H} : \mathbf{x} \times \theta \mapsto \mathbf{u}$, where $\theta$ denotes the set of trainable weights and biases. We consider $\mathscr{H}$ to be a fully connected feed-forward neural network (enclosed in the blue box of Fig. 1), predicts the output $(\hat{u}, \hat{v}, \hat{w}, \hat{p}, \hat{k}, \hat{\varepsilon})$ for a domain point in $(x, y, z)$. Thus, $\hat{\mathbf{u}}(\mathbf{x}; \theta)$ is the approximate solution of the PDEs under consideration. The partial derivatives in the governing equations (e.g., $\partial\hat{\mathbf{u}}/\partial\mathbf{x}$ or higher-order terms) are computed using automatic differentiation. The value of turbulent viscosity [Eq. (2b)] or momentum source term [Eq. (5)] are computed directly from the model outputs. Boundary conditions are enforced as soft constraints through dedicated BC loss terms included in the composite training objective, consistent with the standard PINN formulation. For Dirichlet boundary conditions, the boundary data are directly compared with the output of the PINN, bypassing automatic differentiation. Neumann conditions require automatic differentiation to compute losses. The data losses are formed by comparing the model prediction with any measurements or simulations at specific spatial locations. As the problem considered is steady-state, initial condition loss terms are not included in the PINN formulation.

The parameters ($\theta$) of the neural network $\mathscr{H}$ are learned by minimizing a loss function $\mathscr{L}_{PINN}$ that comprises the PDE-residual loss, boundary loss, and data loss, i.e.,

$$\mathscr{L}_{PINN} = \alpha_b \mathscr{L}_{boundary} + \alpha_r \mathscr{L}_{residual} + \alpha_d \mathscr{L}_{data}, \tag{7a}$$

$$\mathscr{L}_{boundary} = \frac{1}{N_{bp}} \left[ \sum_{i=1}^{N_{bp}} \left( \mathscr{B}(\hat{\mathbf{u}}(\mathbf{x_i})) - b \right)^2 \right], \tag{7b}$$

$$\mathscr{L}_{residual} = \frac{1}{N_c} \left[ \sum_{i=1}^{N_c} \alpha_1 (f_c)^2 + \alpha_2 (f_m)^2 + \alpha_3 (f_k)^2 + \alpha_4 (f_\varepsilon)^2 \right], \tag{7c}$$

$$\mathscr{L}_{data} = \frac{1}{N_{dp}} \left[ \sum_{i=1}^{N_{dp}} \left( \mathscr{M}(\hat{\mathbf{u}}(\mathbf{x_i})) - m \right)^2 \right]. \tag{7d}$$

Here, the boundary conditions are expressed as $\mathscr{B}(\mathbf{u}(\mathbf{x})) = b$ on $\mathbf{x} \in \partial_b\Omega$, and measurement data as $\mathscr{M}(\mathbf{u}(\mathbf{x})) = m$ on $\mathbf{x} \in \partial_m\Omega$, where $\partial_b\Omega$ is the boundary of the domain, $b$ is the boundary condition value, $\partial_m\Omega$ denotes the spatial locations where measurement/simulation data ($m$) is available. $\alpha_b$ is the weighting coefficient for boundary losses, $\alpha_r$ is the weighting coefficient for residual losses, $\alpha_d$ is the weighting coefficient for data losses, and $\alpha_{1-4}$ are the weighting coefficients assigned to the residuals of each governing PDE. The values of these coefficients are fixed based on hyperparameter sensitivity experiments (See Appendix A). The coefficients $\alpha_b$ and $\alpha_d$ are weighed $10\times$ more compared to the coefficient $\alpha_r$ and the coefficients $\alpha_{1-4}$ are given weights in the ratio $100 : 10 : 1 : 1$. $N_c$ represents the total count of location points within the domain, $N_{bp}$ denotes the total number of boundary points and $N_{dp}$ denotes the total number of measurement/simulation data points. The boundary loss and data loss terms are determined using the mean squared error on the provided boundary data and measurement/simulation data, respectively. The PDE-residual loss is calculated as the L2 norm of the residuals at training points calculated using automatic differentiation. Since PDE-residuals are incorporated in the loss function during training, these models are termed physics-informed. (Convergence of the individual loss components during training is shown in Appendix B.)

### C. Implementation

TensorFlow software package[66] is used to implement the PINN model described above. The model has three inputs, six outputs, and other internal layers as shown in Fig. 1. The implementation in the present paper is built inspired by a recent development of a single-turbine PINN model.[67] Table I presents the important hyperparameters such as neurons per layer and sampling points that are determined through extensive experiments (see Appendix C for ablation study). The Swish activation function is used for three hidden layers with 700 neurons each, and the six pre-output layer with 500 neurons,[68] while Rectified Linear Unit (ReLU) is used for the last output nodes representing turbulent kinetic energy ($k$) and dissipation rate ($\varepsilon$) to avoid negative values; other outputs do not have an activation function. Parameter optimization involves the Adam optimizer with adaptive gradient descent and momentum,[69] starting with an exponentially decaying learning rate of 0.0001 at a decay rate of 0.9 and a decay step of 5000. Parameters are initialized using the Glorot uniform initializer.[70] Mini-Batch training is used for the three-dimensional wake simulation due to the high collocation points, performed on an NVIDIA RTX A6000 GPU. The code, the model architecture, and the trained weights have been released on GitHub with a Zenodo DOI.[71]

## III. THREE-DIMENSIONAL TURBULENT FLOW SIMULATION ON A GRID OF WIND TURBINES

The PINN model is utilized to forecast the wake generated and the power output in a multi-turbine wind farm grid. Wakes occur downstream due to the obstruction created by the wind turbine as it disrupts the incoming airflow. All turbines are represented using an actuator disk model[72,73] and incorporated into the $k-\varepsilon$ RANS model as a source term (Sec. II A). The domain and experimental setup are discussed in Sec. III A. For the purpose of numerical simulations, the wake is computed in a grid configuration with four turbines having specifications of the Sexbierum HOLEC WPS 30/3 Wind Turbine. Section III C discussed the hybrid training strategy for the PINN model, followed by the simulations in Sec. III D.

### A. Domain and boundary conditions

Figure 2 shows the schematic of the domain used for simulations with a rectangular grid pattern. We perform simulations on a rectangular grid pattern of wind turbines. Wind turbines (WTs) are

**TABLE I.** Hyperparameters used for wind turbine wake simulation.

| Neurons (hidden layers) | 700 |
|---|---|
| Neurons (output layer) | 500 |
| Learning rate | 0.0001 |
| Learning rate scheduler | Exponential decay |
| Epochs | 15 000 |
| Optimizer | Adam |
| Trainable-parameters | 3 090 200 |
| Batches | 16 |
| Activation function | Swish + ReLU |
| Initializer | Glorot uniform |

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

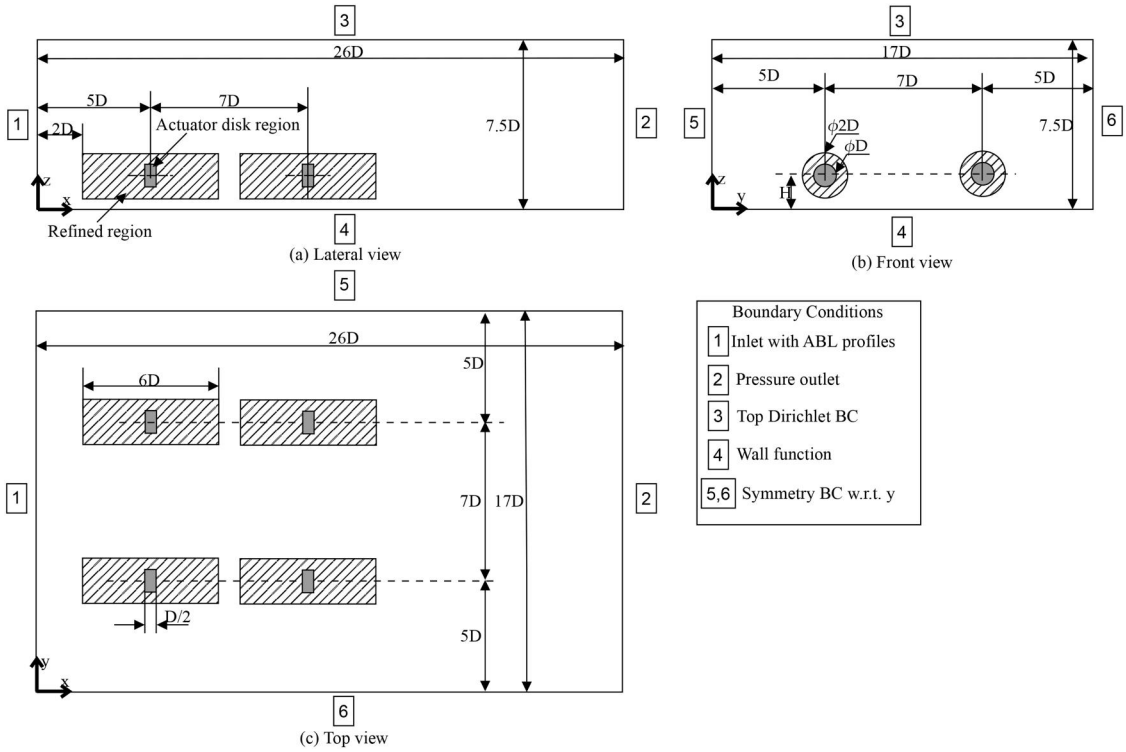



**FIG. 2.** Schematic of the rectangular grid domain of wind turbines: (a) lateral view, (b) front view, and (c) top view. D is the diameter of the turbine, and H is the hub height. The boundary conditions used on the domain surfaces are also shown.

modeled according to the specifications of the Sexbierum HOLEC WPS 30/3 Wind Turbine. The HOLEC WPS 30/3 WT has a diameter D = 30.1 m and a hub height H = 35 m. The boundary conditions are set for a free stream wind speed ranging from 7 to 11 m/s with a turbulence intensity of 10% at the hub height. The thrust coefficients corresponding to these conditions are also used from the thrust curve. Figure 3 shows the power curve and the thrust coefficient curve of the HOLEC WPS 30/3 WT. The PINN model relies solely on the thrust coefficient curve for the training phase, whereas the power curve is utilized during the post-processing stage. For each simulation, the PINN model is trained from scratch.

Fully developed inlet profiles are specified at the inlet boundary for mean velocity [Eq. (8a)], turbulent kinetic energy [Eq. (8b)] and turbulent dissipation rate [Eq. (8c)] as follows:[74]

$$u = \frac{u_\tau}{\kappa} \ln\left(\frac{z + z_0}{z_0}\right), \tag{8a}$$

$$k = \frac{u_\tau^2}{\sqrt{C_\mu}}, \tag{8b}$$

$$\varepsilon = \frac{u_\tau^3}{\kappa(z + z_0)}, \tag{8c}$$

where $u_\tau = \sqrt{\frac{\tau_w}{\rho}}$ is the friction velocity (with $\tau_w$ is the wall shear stress, and $\rho$ is the fluid density), $\kappa \approx 0.418$ is the von Kármán constant, $z$ is the height coordinate, and $z_0$ is the aerodynamic roughness height. Inlet profiles [Eq. (8)] are analytical solutions of the momentum and $k - \varepsilon$ model equations.[75] At the outlet, a provided pressure boundary condition and zero gradients along the streamwise direction for other quantities are imposed. At the bottom wall/ground, wall functions consistent with the inlet profiles are used to specify Dirichlet conditions for velocity, turbulent kinetic energy, and

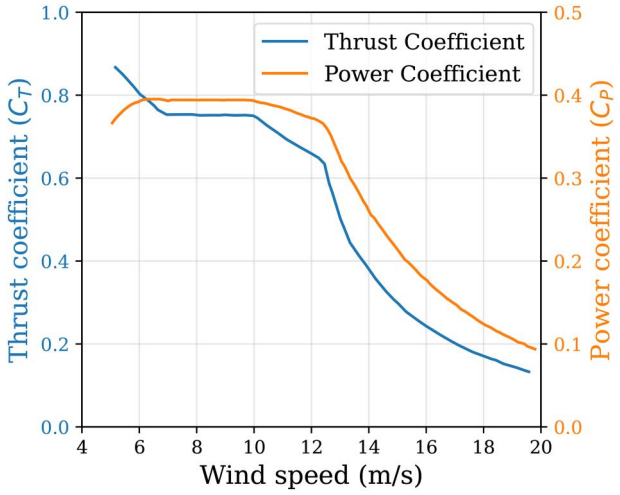



**FIG. 3.** Thrust and power coefficient curves of the HOLEC WPS 30/3 WT.

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

dissipation rate. Dirichlet conditions are also applied on the top face to stay consistent with the inlet profiles. The lateral faces in the three-dimensional domain are provided with symmetry boundary conditions with a zero velocity normal to the face.

### B. Sampling of boundary and domain collocation points for training

For the training of the PINN model, the boundary loss [Eq. (7b)] and PDE-residual loss [Eq. (7c)] are evaluated at points sampled using the grid-based method and the Hammersley method,[76,77] respectively. Figure 4 shows the sectional visualization of the three-dimensional computational domain used for the flow simulations. The orange markers denote the boundary points, while the blue and green markers represent the domain collocation points used only for evaluating PDE-residual losses. The green regions correspond to the actuator disk zones that model the turbine rotors. The source terms used to model the thrust force are applied in a cylindrical region with a diameter equal to the turbine diameter (D) and a length of 0.25 times the turbine diameter upstream and downstream of the disk.[65] The inset in Fig. 4 provides an enlarged view of one actuator disk region, where the thrust force is normally distributed along the streamwise direction, allowing a seamless calculation of PDE-residual losses by avoiding a sudden change in the disk region. Downstream of the wind turbine are two distinct regions: near-wake and far-wake.[78] The near-wake region, which extends two to four times the turbine diameter downstream, features a complex flow influenced by the rotor design and the aerodynamics of the blade.[79,80] To accurately capture near-wake behavior, the domain collocation points for training are increased within a cylindrical volume of length six times the turbine diameter and diameter two times the turbine diameter, symmetrically placed around the disk as indicated by the refined region in Fig. 2 and by the dense clustering of blue points near turbines in Fig. 4. This method is inspired by traditional numerical solver mesh refinement practices. In the far-wake region, only thrust force and turbulent kinetic energy are affected by the turbine, thus necessitating fewer sampling points than in the near-wake region. (Total number of sampled points are discussed in Appendix B. Three-dimensional illustration in provided in Appendix A.)

### C. Hybrid physics and data-driven training strategy

The nature of the wake generated behind the turbines depends on the incoming wind. Various methods have been reported to estimate the characteristics of the incoming wind in a wind farm,[81,82] such as using wave wind Light Detection and Ranging (LiDAR),[83] wave Doppler LiDAR,[84,85] and anemometer towers.[86] These measurements are mostly restricted to sparse locations, such as line-of-sight wind observations along the beam path in LiDAR, which prevents the direct reconstruction of the detailed spatiotemporal wind field.[87] Therefore, coupling the sparse data with flow physics is a suitable strategy for estimating the complete wind field, and the use of such measurement to improve PINN training is illustrated in this study by using synthetic measurement data drawn from the RANS solution computed by conventional CFD solvers. In this study, the term "sparse data" denotes data that are sparse in three-dimensional space (available only on a single horizontal plane) and additionally of low resolution on that plane (i.e., coarsely sampled). A two-dimensional slice extracted from the three-dimensional CFD simulation is combined with the governing PDEs to train the PINN model within our hybrid physics–data-driven training strategy. Specifically, the streamwise velocity at the domain points lying on the horizontal plane through the turbine center is employed for training. The numerical solver with the chosen mesh size produces 193 776 grid points on this horizontal plane. For PINN training, however, we utilize a significantly coarser grid containing only 3876 points, which is 50× fewer than the grid points used in the numerical solver. The use of a regularly spaced, grid-based dataset is not mandatory; a randomly

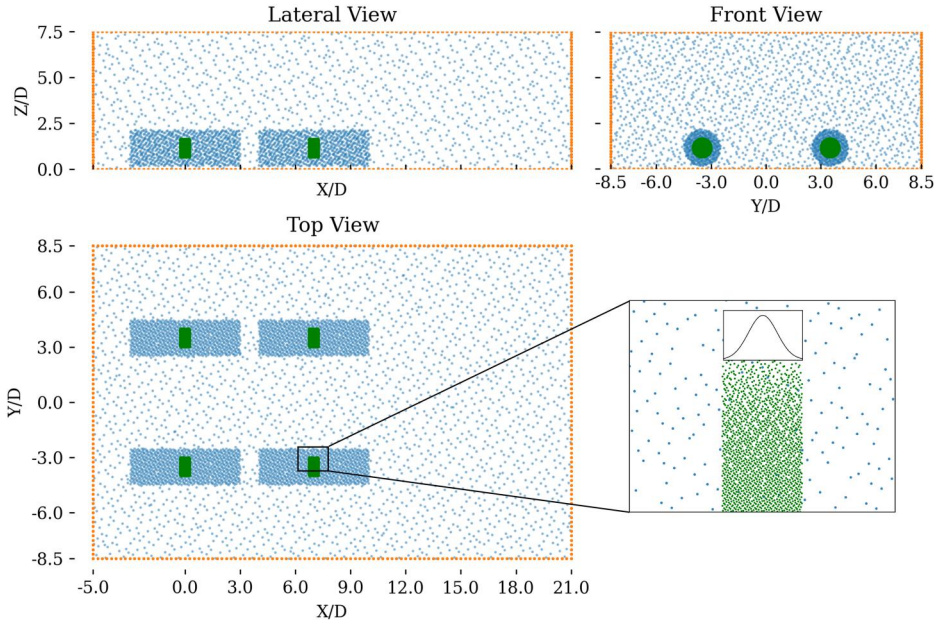



**FIG. 4.** Sectional views of the three-dimensional domain. (i) Orange: boundary points, (ii) blue: domain collocation points. Dense region indicates the refinement near the turbine, and (iii) green: actuator disk region. The inset shows an enlarged view of the region near the actuator disk. The thrust force is distributed normally along the streamwise direction. The axes are normalized by the turbine diameter (D).

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

**TABLE II.** RMSE between the simulation from the PINN model and the numerical solver. Normalized velocity and TKE are compared at the inlet, 3D, 6D, 9D, 12D, and 15D downstream of the turbine in a horizontal plane at the hub height of the wind turbines.

| | Inlet | 3D | 6D | 9D | 12D | 15D |
|---|---|---|---|---|---|---|
| Normalized velocity | 0.001 19 | 0.002 17 | 0.002 07 | 0.002 40 | 0.002 08 | 0.001 82 |
| Normalized TKE | 0.004 73 | 0.389 26 | 0.402 84 | 0.481 42 | 0.502 01 | 0.424 80 |

distributed dataset can also be employed during training (see Appendix B for details). Figure 11(a) presents the simulation results when observational data are incorporated alongside the PDEs.

A simulation based on the finite volume method is executed in Simcenter StarCCM+[88] using the Shear Stress Transport (SST) $k-\omega$ turbulence model.[89] The turbulence model and the mesh size used were selected on the basis of related studies to ensure a mesh-independent solution.[42,90] This simulation data are used as a sensor simulator and ground truth for verification of the PINN model simulations.

### D. Wake simulations from the trained PINN model

Tables II and III and Figs. 5–7 present the main results of the PINN simulation. The goodness of the PINN simulation is computed by calculating the error [as Root Mean Squared Error (RMSE)] between the PINN simulation and the corresponding StarCCM+ CFD simulation on two planes.

Figure 5 shows the contour plot of the RANS-PINN turbulent flow simulations (velocity, turbulence intensity and pressure) in horizontal and vertical planes, passing through the hub of the wind turbines for the rectangular grid. Figures 6 and 7 show the normalized velocity and normalized turbulent kinetic energy profiles in two different planes at the inlet and different downstream locations (3D, 6D, 9D, 12D, and 15D, where D is the turbine diameter) of the first turbine along with the simulation from the CFD k–$\omega$ SST solver for comparison. The flow variables are normalized by the corresponding far-field values from the simulation. The Root Mean Square Error,

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\phi_i^{\mathrm{PINN}} - \phi_i^{\mathrm{CFD}}\right)^2},$$

is evaluated, where $\phi$ represents the normalized flow variable (either velocity or TKE), $N$ is the number of sampling points at a specified downstream location, and the superscripts $(\cdot)^{\mathrm{PINN}}$ and $(\cdot)^{\mathrm{CFD}}$ indicate, respectively, the prediction of the PINN model and the reference CFD solution obtained with the $k$–$\omega$ SST model. These RMSE values are computed for simulations of the PINN model carried out over multiple realizations with different random seeds (see Appendix E). The RMSEs for the normalized velocity and TKE at different locations in horizontal and vertical planes along the height of the hub are reported in Tables II and III, respectively. Thus, qualitatively and quantitatively, we see that the developed PINN model is capable of giving results comparable to the SST k–$\omega$ model. A study on different turbulence models had concluded that the SST k–$\omega$ model is an effective turbulence model for wind turbine simulations.[90] Thus, the fact that our PINN model matches well with the simulations of SST k–$\omega$ gives us confidence to deploy a PINN forward simulator as a wake surrogate for downstream applications.

In the contour plot (Fig. 5), the wake created by the turbine is evident from the low-velocity (panel a) and low-pressure (panel c) regions downstream of the turbines. This causes turbulent transport of momentum from the ambient wind into the wake region as seen in the turbulence intensity contour plot (Panel b). The wind turbine generates turbulence in addition to that already produced by the terrain, indicated by the increased turbulence intensity behind the turbine. It is also evident that the turbulence intensity has a sharp structure emerging from the disk edges which is an indication of vortex formation immediately behind the rotor blade tips (marked by numeral I). Further downstream, a shear layer is formed and encloses the wake (marked by numeral II). The far wake region is also well predicted by the PINN model. It is indicated by the expansion of the shear layer to the center by turbulent diffusion (marked by numeral III). The axisymmetric nature of the wake seen in the contour plots is a result of the RANS closure being used for the simulations. The RANS turbulence closure fails to simulate highly transient fine vortex tubes, which in turn obstruct the development of complex vortex structures, and the simulated vortex tube moves in a straight line without meandering. The pressure jump across the disk is seen from the pressure contours (marked by numeral IV), which is also an indication of the thrust force exerted by the turbine on the incoming flow. A higher pressure jump indicates a larger thrust force.

Figure 8 shows the comparison of velocity and turbulence intensity between PINN model and CFD solver in a horizontal plane, along with the absolute error map. The velocity magnitude indicates that the PINN model effectively reproduces the wake structures and

**TABLE III.** RMSE between the simulation from the PINN model and the numerical solver. Normalized velocity and TKE are compared at the inlet, 3D, 6D, 9D, 12D, and 15D downstream of the turbine in two vertical planes through the center of the wind turbines at $\pm 3.5D$.

| | y/D | Inlet | 3D | 6D | 9D | 12D | 15D |
|---|---|---|---|---|---|---|---|
| Normalized velocity | −3.5 | 0.032 27 | 0.044 03 | 0.045 13 | 0.047 40 | 0.051 25 | 0.055 45 |
| | 3.5 | 0.034 64 | 0.048 03 | 0.048 41 | 0.044 87 | 0.049 56 | 0.055 94 |
| Normalized TKE | −3.5 | 0.025 32 | 0.359 19 | 0.329 36 | 0.499 95 | 0.503 44 | 0.480 02 |
| | 3.5 | 0.028 99 | 0.406 95 | 0.349 70 | 0.510 40 | 0.487 27 | 0.460 44 |

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

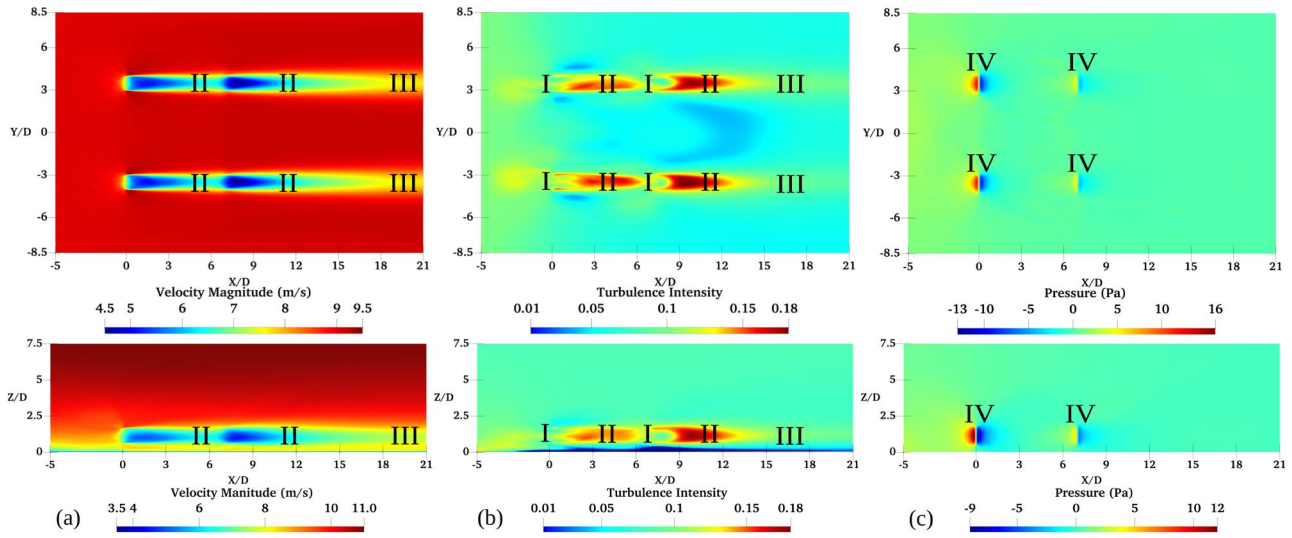


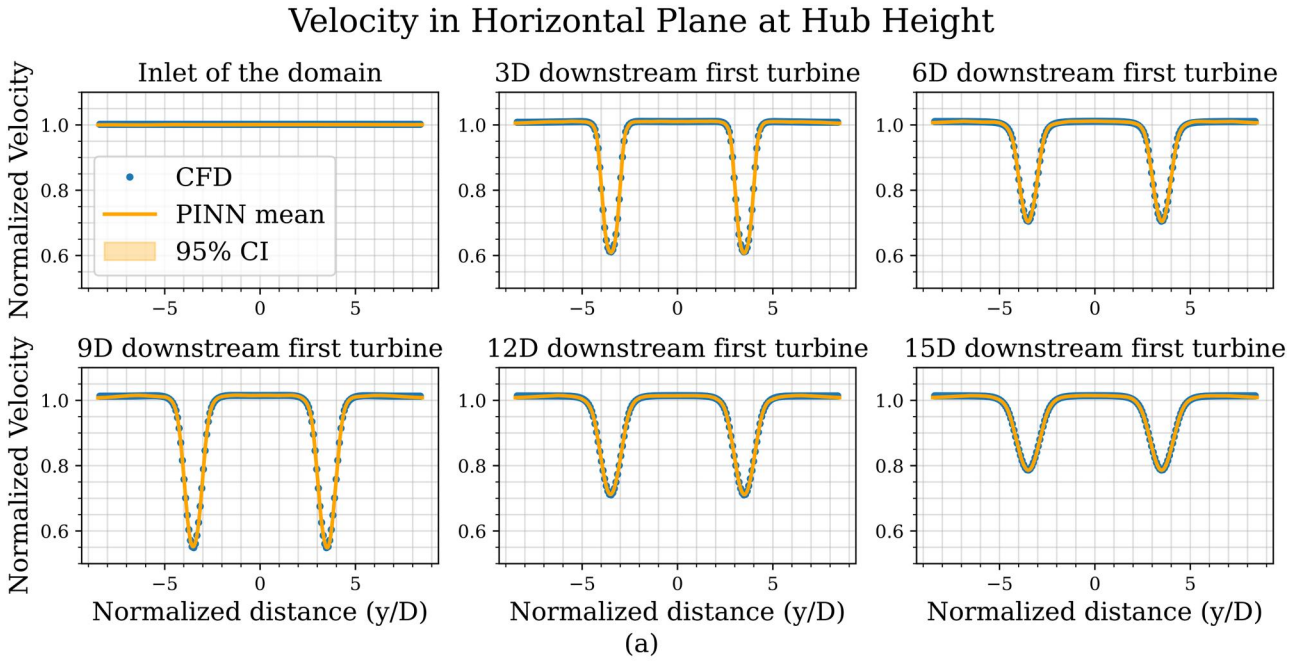

**FIG. 5.** Contours of (a) velocity, (b) turbulence intensity, and (c) pressure in two planes through the center of wind turbines in a rectangular grid for $U_{ref} = 9$ m/s: (i) top: horizontal plane and (ii) bottom: vertical plane at y/D = ±3.5.

velocity deficits downstream of the turbines, demonstrating close agreement with the CFD results. The turbulence intensity exhibits similar spatial trends, with elevated values observed within the wake regions. The absolute error maps reveal that the deviations are primarily confined to the wake shear layers, where strong velocity gradients and turbulence fluctuations occur (Error map in vertical plane is shown in Appendix F). Overall, the PINN model accurately captures the flow physics, particularly the variations in velocity. Figure 9 also substantiates the findings, which show the streamlines around the wind turbine in horizontal and vertical planes for simulations from the PINN model and the CFD solver. Flow variations near the turbine are well simulated in both planes. Even though no data are used in the vertical plane, contours and streamlines below and above the turbine [Fig. 9(b)] indicate that the PINN model effectively learns the physics using the hybrid training strategy.

Further comparison with the CFD numerical simulation is performed to validate our PINN model at different downstream locations of the wind turbine at a distance of 3D, 6D, 9D, 12D, and 15D in both vertical and horizontal planes through hub of the wind turbines. The profiles at the inlet are also compared. The PINN predictions shown in Figs. 6 and 7 correspond to the ensemble mean obtained from five independently trained models with different random initializations, while the shaded regions represent the 95% confidence intervals computed from the ensemble predictions, illustrated as shaded bands. These confidence intervals provide an estimate of the predictive uncertainty of the PINN model. Figures 6(a) and 7(a), respectively, show the normalized wind speed as a function of the spanwise direction in a horizontal plane and as a function of wall-normal direction in a vertical plane through the hub of the wind turbine for the rectangular grid. The center or hub of the four wind turbines in the rectangular grid is located at ± 3.5D in the spanwise direction at 0D and 7D in the streamwise direction. The maximum wind speed deficit is observed in the direction behind the center of the turbine, as expected. The quantitative comparison shows that the downstream wind speed predicted by the model aligns well with the CFD solver in both planes and remains within the estimated confidence intervals at most downstream locations, indicating that the PINN surrogate delivers trustworthy predictions with appropriately quantified uncertainty. Figure 7(a) shows the effectiveness of the hybrid training strategy to employ PINN as a forward solver with sparse data availability. For our hybrid training strategy, only data for a single flow variable ($u$) in the horizontal plane are used. However, the velocity profiles in the vertical plane also match well with the CFD solver showing that the PINN model has learned the dynamics accurately. Figures 6(b) and 7(b) show the normalized turbulent kinetic energy at the above-mentioned downstream locations in the horizontal and vertical planes, respectively. In the near-wake region of the turbines (3D and 9D), the PINN model is able to predict the TKE peaks generated by the tip vortices similar to the SST $k-\omega$. It is more evident in 9D that it is closer to the turbine. A distorted peak formation is observed close to the second row turbine (6D). Further downstream (12D and 15D), the TKE profiles exhibit a flatter peak. As expected, slightly larger uncertainty is likewise detected in the wake region downstream of the turbines, where the flow dynamics exhibit increased complexity. In addition, the RMSE for TKE reported in Tables II and III appears higher compared to the velocity components. This discrepancy arises primarily from the modeling inconsistency between the reference CFD solver and our PINN formulation. In our work, the PINN is trained using the k–$\varepsilon$ turbulence model, whereas the reference CFD data are generated using the SST k–$\omega$ model. These two RANS closures are known to produce systematically different TKE distributions, particularly in regions with strong shear, and wake recovery zones. As a result, even when the velocity fields match closely, the TKE fields exhibit inherent model-form discrepancies that inflate error metrics.[89,91,92] Although the TKE is slightly underestimated between the two columns of the turbines in the far downstream regions, overall, the PINN model provides simulations that are able to predict

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing



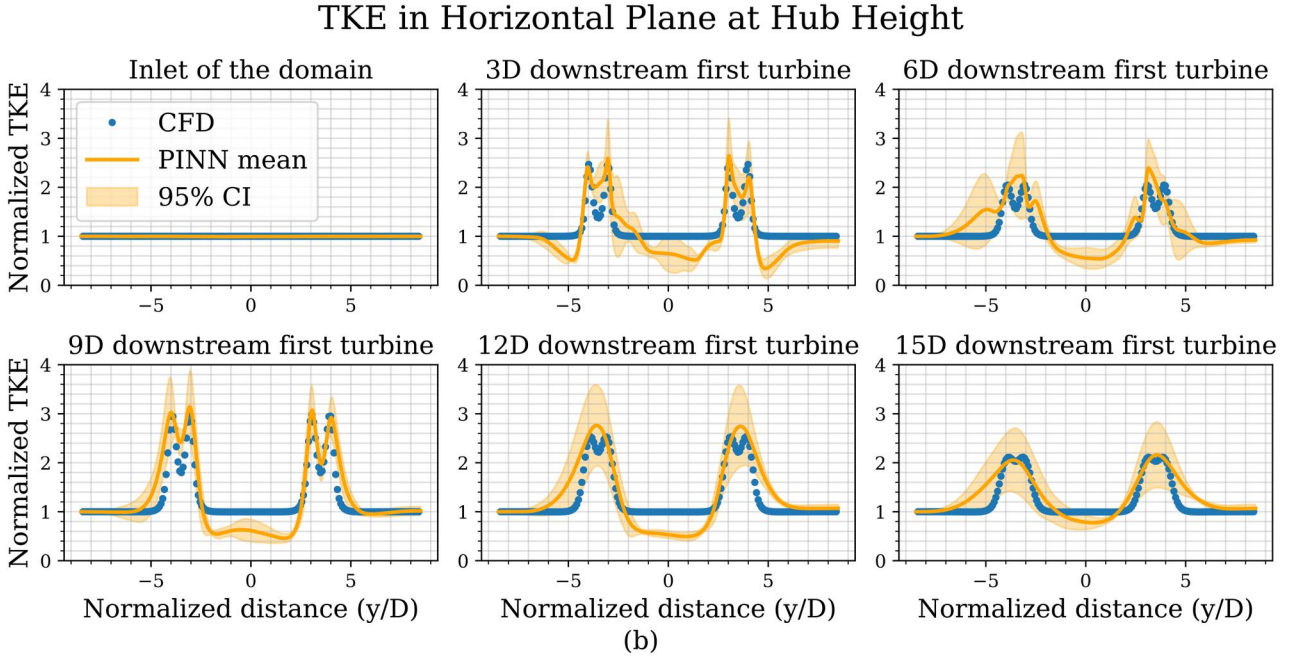




**FIG. 6.** Comparison of (a) velocity and (b) turbulent kinetic energy predicted by the PINN model with the numerical solver in a horizontal plane at the hub height of the wind turbines. The PINN mean predictions (solid orange) with 95% confidence intervals (shaded) are shown alongside CFD reference results (blue dots).

the peaks with good precision with the aid of limited velocity data. Similar results for another grid configuration are discussed in Appendix G. All simulations shown in this section are based on $U_{ref} = 9\,\mathrm{m/s}$. Simulations on the rectangular grid for other reference velocities are shown in Appendix H.

### E. Power output from the grid

In a wind farm design problem, a quantity of interest is the power output from each turbine. The power output of the individual turbines on the grid is calculated using Eq. (4) with the velocity at 3D upstream of the turbine and the corresponding power coefficient (Fig. 3). Figure

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

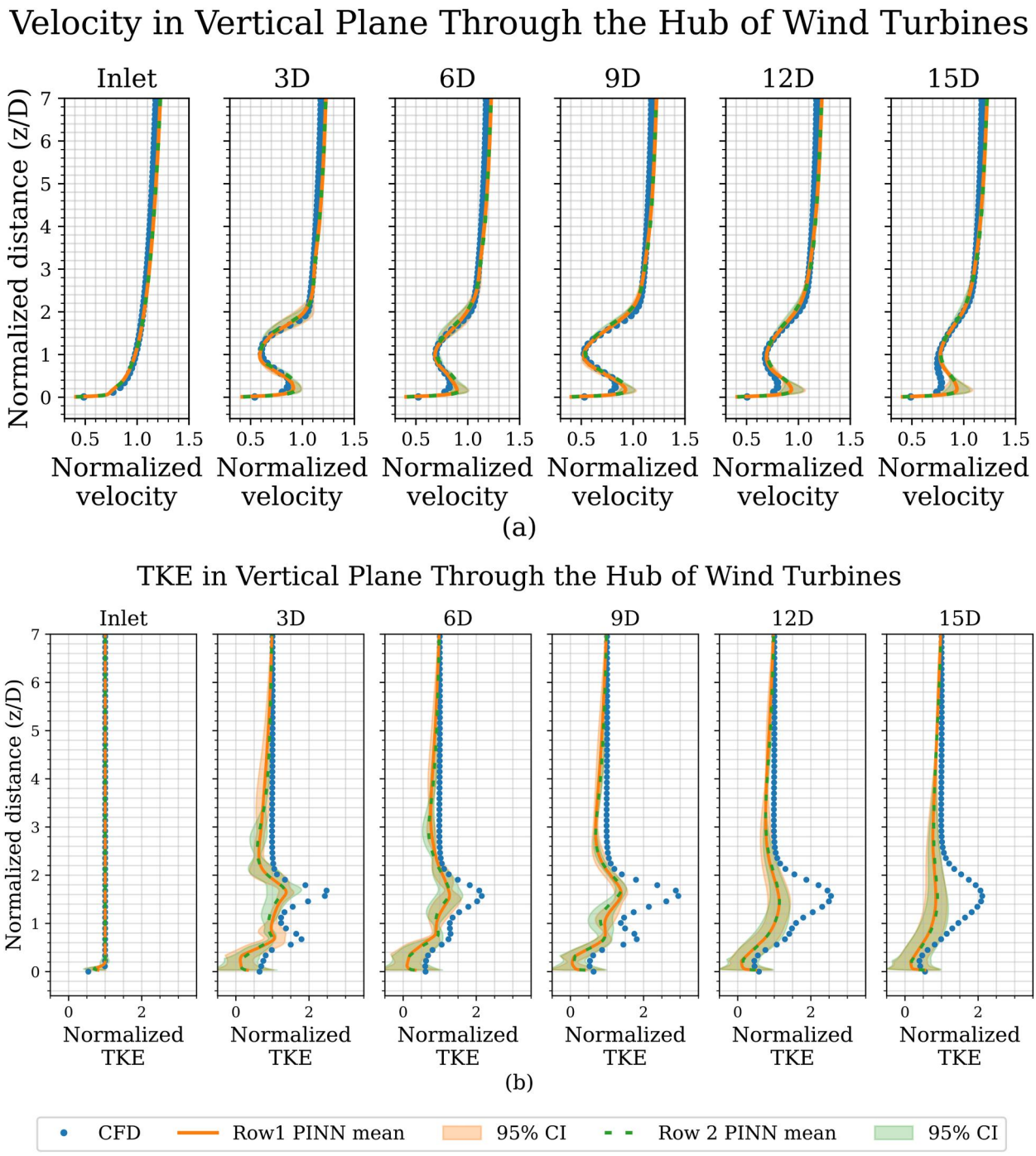



**FIG. 7.** Comparison of (a) velocity and (b) turbulent kinetic energy predicted by PINN model with the numerical solver in a vertical plane through the center of wind turbines. PINN mean predictions for Row 1 (solid orange) and Row 2 (dashed green) with 95% confidence intervals (shaded) are shown alongside CFD reference results (blue dots).

10(a) shows the power output of turbines placed at different downstream locations calculated by the velocity predictions of the PINN model. Power calculated from the CFD solver is also shown for comparison. The absolute error and percentage error between the power outputs predicted by the PINN model and the CFD solver is reported in Table IV. Percentage errors remain under 0.2% for Row 1 turbines across all inlet velocities, and under 0.4% for Row 2 turbines, with the exception of the 11 m/s case, which exhibits a 1.7% error. These results

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

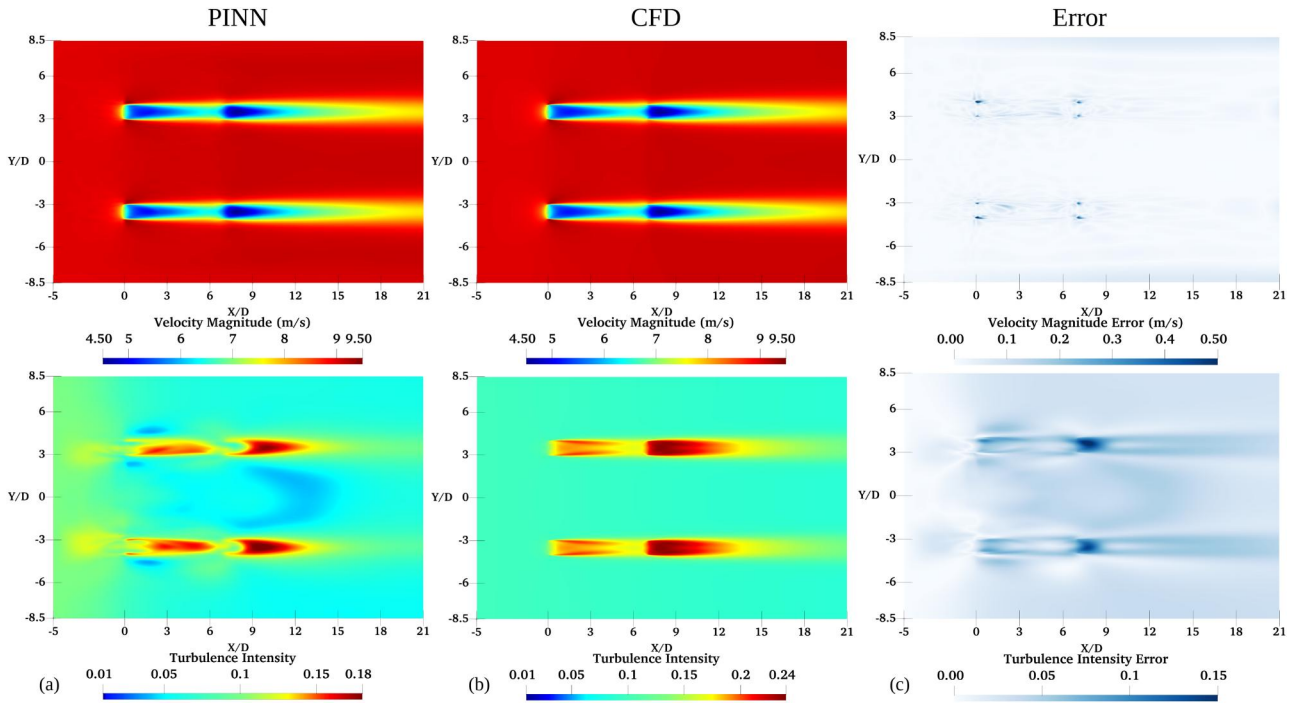



**FIG. 8.** Contours showing comparison of velocity and turbulence intensity between PINN model and CFD solver in a horizontal plane at the hub height of the wind turbines in a rectangular grid for $U_{ref} = 9\,\mathrm{m/s}$. (a) PINN, (b) CFD, and (c) error map: (i) top: Velocity magnitude and (ii) bottom: turbulence intensity.

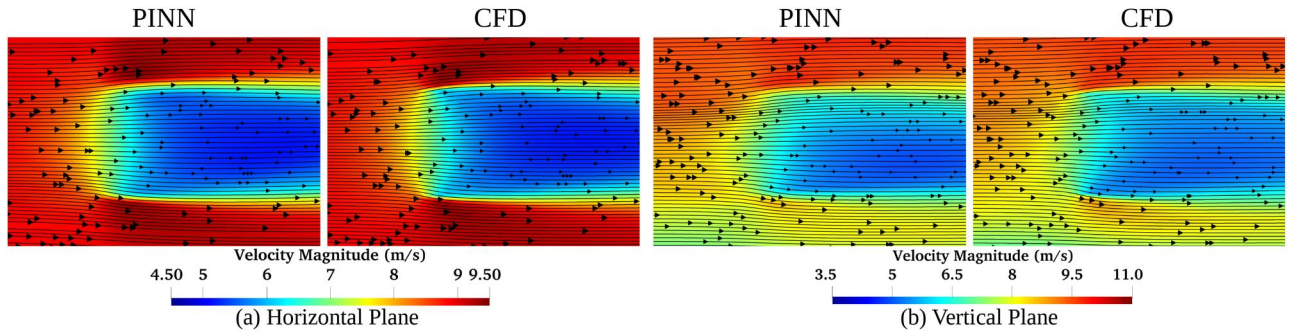



**FIG. 9.** Contours of velocity magnitude along with streamlines near the first wind turbine for simulations from PINN model and CFD solver in (a) horizontal plane and (b) vertical plane for $U_{ref} = 9\,\mathrm{m/s}$.

indicate that the PINN model provides highly accurate predictions of power output across the examined operating range. The plots and the error values show that the PINN model is able to give an excellent estimate of the power output. A velocity difference of typically 1% can result in a power deviation of >2.5% due to the cubic relationship between velocity and power. When normalizing the power output with the values of the first turbine, it can be seen that the normalized values of the second row turbine are almost the same across different inlet velocities [Fig. 10(b)]. This also indicates that the wake simulated by the RANS turbulent models at different velocities exhibits a similar nature, giving further confidence in deploying a PINN model as a forward simulator.

## IV. DISCUSSION

### A. Issues faced while training with only PDE-residual loss

Before using measurement data simulated from a CFD solver for training the PINN model, we tried simulating the wake generated on a grid of turbines without using any simulated measurement data, using only the PDE-residual loss term. However, the simulations of the PINN model, shown in Fig. 11(b), indicate that the prediction does not accurately reflect the real world system intended to be modeled. On close observation, it is evident that the momentum source term [Eq. (5)] is overestimated, causing a pulling effect on the flow.

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

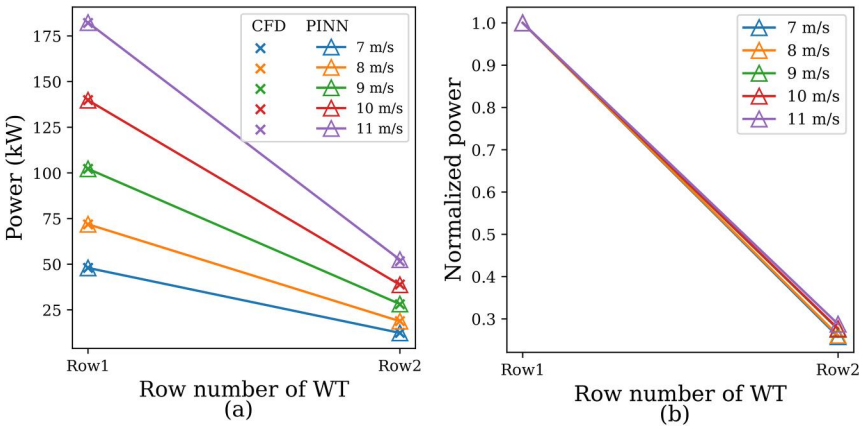



**FIG. 10.** Comparison of (a) power output and (b) normalized power output calculated by the velocity predictions of the PINN model for wind turbines located at different downstream locations for different inlet velocities. Power output predicted by the CFD solver is also shown in (a).

For applications with only a single turbine case, the velocity to calculate the source term could be provided with a static upstream value. However, in situations where there are turbines downstream of the wake, the velocity will vary until a steady state is reached. Our training experiments show that the PINN model fails to reach an optimum where the calculated thrust force achieves a balance with the boundary conditions of the domain, causing an increase in the streamwise velocity upstream of the turbines. Although the contours of pressure, spanwise velocity, and wall-normal velocity exhibit a trend close to the reality, the turbulent flow parameters are simulated very poorly. To assert the findings, we tried to limit the magnitude of the source term by clipping the maximum value of the streamwise velocity for the source term calculation. We saw an improvement in the simulations [Fig. 11(c)]. However, the estimated values of source terms always touch the maximum value. The variation of mean squared values of the source term and x component of the momentum equation during the training is shown in Fig. 12. It can be observed that, when trained only with PDE residuals, the value of the source term varies throughout the training, indicating that the model does not reach an optimal value for the source term. Whereas the other two training strategies help the model to stabilize the value of the source term after a few training epochs. However, limiting the magnitude of the source term affects the model's ability to minimize the loss in the x-momentum equation. At the end of the training, the mean squared values of the source term are 4.0924, 8.4607, and 5.8588 for three training strategies: (i) hybrid-training (PDE + Data), (ii) with only PDE residuals, and (iii) limit the magnitude of the source term, respectively. The respective values of the x-momentum equation are 0.0353, 0.0609, and 0.9836. It is also evident from Fig. 11(d), which shows the quantitative comparison of the velocity at different downstream locations in the horizontal plane, that the hybrid training strategy provides a substantial improvement in the simulations. These findings suggest that some kind of regularization or control is needed for the PINN model to successfully simulate the domain with a turbine grid. (The importance of enforcing physical constraints with sparse data are further demonstrated in Appendix J.)

## B. Scalability of the model

To evaluate the scalability of the proposed PINN-based surrogate model, we extended our simulation setup from a 4-turbine configuration to a more complex 6-turbine layout. The transition to a larger wind farm scenario increases the spatial domain size, the number of actuator forcing regions, and the complexity of wake interactions. Despite this added complexity, the model maintained stable training behavior and produced accurate flow-field predictions. It can be observed from the Fig. 13 that the model scales effectively with the number of turbines. In particular, the increase from 4 to 6 turbines required only a modest increase in computational cost while preserving prediction quality. This indicates that the underlying architecture and training strategy generalize well to larger multi-turbine configurations.

## C. Training and inference time

We utilize a single NVIDIA RTX A6000 GPU with 48GB of memory on a dedicated workstation with an Intel Core i9 CPU operating at 3.70 GHz for training the PINN model. We trained the three-

**TABLE IV.** Absolute and percentage error in power output calculated from the predictions of the PINN model compared to the CFD k–$\omega$ SST numerical solver for wind turbines located at Row 1 and Row 2 for inlet velocities ranging from 7 to 11 m/s. Absolute errors are reported in kW. Percentage error is defined as $\frac{|P^{\text{PINN}} - P^{\text{CFD}}|}{P^{\text{CFD}}} \times 100$, where $P^{\text{PINN}}$ and $P^{\text{CFD}}$ denotes the PINN and reference CFD power outputs, respectively.

| Velocity | 7 m/s | | 8 m/s | | 9 m/s | | 10 m/s | | 11 m/s | |
|---|---|---|---|---|---|---|---|---|---|---|
| Row | 1 | 2 | 1 | 2 | 1 | 2 | 1 | 2 | 1 | 2 |
| $P^{\text{PINN}}$ (kW) | 47.9567 | 12.3726 | 71.8642 | 18.7750 | 102.2254 | 28.1674 | 139.7971 | 38.7257 | 182.3285 | 52.4613 |
| $P^{\text{CFD}}$ (kW) | 47.9623 | 12.3594 | 71.7752 | 18.7931 | 102.1894 | 28.0234 | 139.8541 | 38.8612 | 182.1030 | 51.6025 |
| Abs. error (kW) | 0.0057 | 0.0132 | 0.0890 | 0.0181 | 0.0361 | 0.0144 | 0.0570 | 0.1355 | 0.2255 | 0.8588 |
| % Error | 0.0119 | 0.1068 | 0.1240 | 0.0963 | 0.0353 | 0.0513 | 0.0408 | 0.3487 | 0.1238 | 1.6643 |

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

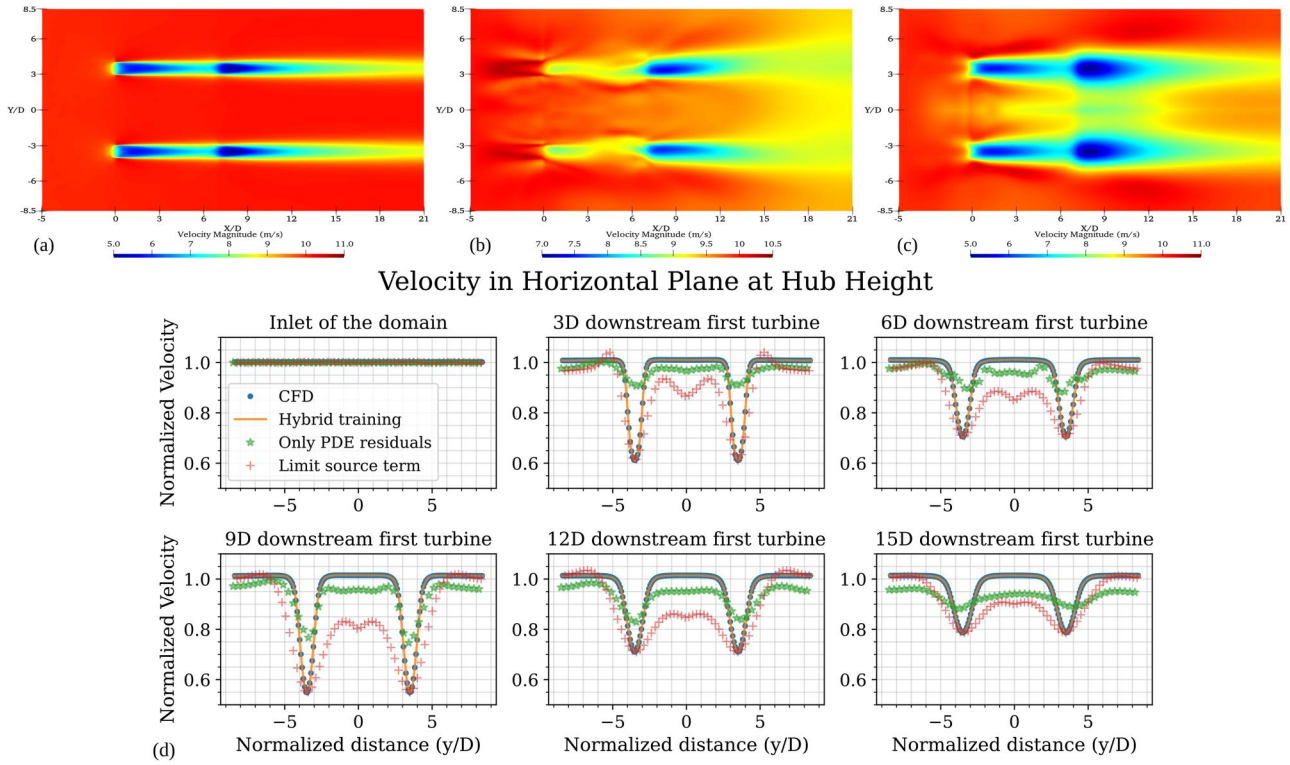



**FIG. 11.** Contours of velocity in a horizontal plane at the hub height of the wind turbines for three PINN training strategies: (a) hybrid-training (PDE + Data), (b) with only PDE residuals, (c) limiting the magnitude of the source term, and (d) comparison of the velocity predicted by the PINN model with the numerical solver for the three training strategies.

dimensional wake simulation for a wind turbine grid with about 61 600 sampling points from the domain, which took 2087.15 ± 9.90 min. The total training time for the PINN model reduced to 1018.98 min when executed on a workstation featuring an NVIDIA RTX PRO 6000 Blackwell GPU with 96 GB memory, highlighting the potential of emerging GPU technologies to significantly shorten computational time. The CFD simulations with the StarCCM+ solver were evaluated using a machine equipped with an Intel Xeon processor featuring 16 physical cores operating at 2.49 GHz. The simulation, based on a high-resolution CFD mesh containing approximately $36 \times 10^6$ cells, was executed using 4 active cores. The total solver elapsed time was 104 385.74 s or 1739.76 min. Inference using the PINN model is much faster and does not need high-performance GPUs. On our desktop with an NVIDIA GeForce RTX 2070, line plot simulations with 1000 spatial points were completed in milliseconds. For contour plots on a 500 × 500 × 1 grid, inference took less than 30 s. A drawback of current PINN frameworks, including ours, is the long training time compared to CFD solvers. However, advancements like the Physics-informed transformer

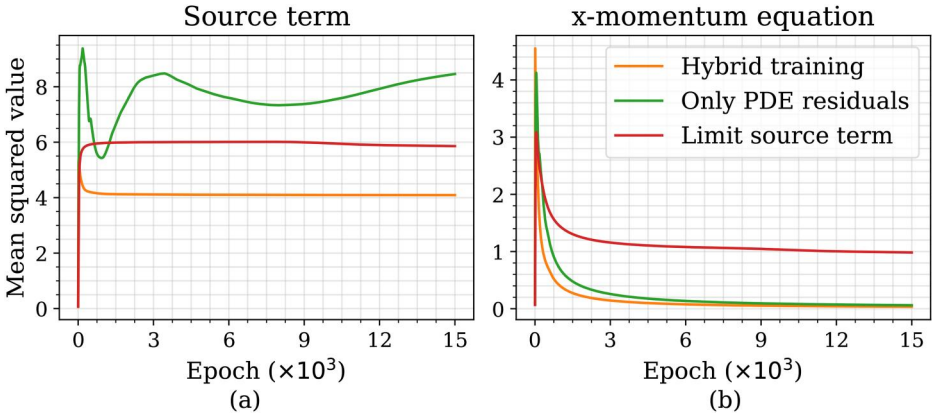



**FIG. 12.** Variation of mean squared values of (a) source term and (b) x component of momentum equation during model training for three training strategies: (i) hybrid-training (PDE + Data), (ii) with only PDE residuals, and (iii) limiting the magnitude of the source term.

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

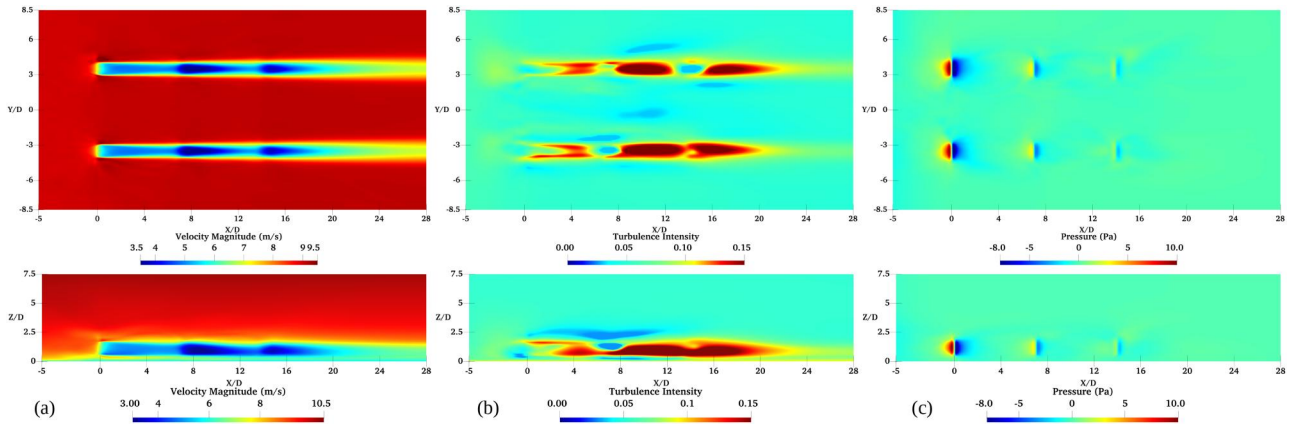



**FIG. 13.** Contours of (a) velocity, (b) turbulence intensity, and (c) pressure in two planes through the center of wind turbines in a rectangular grid of six turbines for $U_{ref} = 9\,\mathrm{m/s}$: (i) top: horizontal plane and (ii) bottom: vertical plane at y/D = ±3.5.

neural operator (PINTO)[93] aiming to generalize boundary and initial conditions, might enable models to train once for diverse scenarios. Our PINN models are distinct as they are trained with limited data of one flow variable and incorporate PDE residual loss, unlike most data-driven neural models that need comprehensive data from the entire domain and have difficulty predicting non-included variables. Thus, with sparse velocity data from simulated LiDAR or anemometer towers, our PINN model provides deep insights into the flow across the domain. Neural networks facilitate transfer learning, using a pre-trained model for one flow scenario as a basis for another, reducing computational requirements during training, as shown next.

### D. Transfer learning for other inlet conditions

Even though the training time of the hybrid PINN model is less than that of the PDE-only model, it is still higher than the run time of a state-of-the-art numerical solver. Hence, we adopt the transfer learning strategy to reduce the training time for new turbulent flow situations. While transfer learning has been explored in PINNs framework,[94,95] the present work demonstrates its effectiveness in the specific setting of multi-turbine wake interactions, where the domain involves multiple actuator disk sources, and sparse measurement data under varying inflow and atmospheric stability conditions. The approach reduces computational requirements when adapting to new operating conditions without retraining from scratch. In transfer learning, a pre-trained model checkpoint is used as the starting point for training models for new tasks/situations. For three-dimensional wake simulation on a grid of wind turbines, changing the reference velocity ($U_{ref}$) or turbulence intensity ($TI_{ref}$) changes the boundary conditions, which affects the thrust and power output, thereby requiring a new training of the PINN model.

In the experiments, we show (i) how a higher-velocity test case can be quickly simulated from a pre-trained lower-velocity PINN model and vice versa while maintaining a constant turbulence intensity across the simulations and (ii) how a test case with different velocity and turbulence intensity can be simulated from a pretrained model with different conditions. First, the PINN model is trained with $U_{ref} = 7 - 11$ m/s at a turbulence intensity of 10% for 15 000 epochs. The 11 and 7 m/s simulations are used as references for the higher velocity case and the lower velocity case, respectively. The other velocity simulations in each test case are the pre-trained model in a transfer learning setting. In transfer learning for the higher velocity case, $U_{ref} = 11\,\mathrm{m/s}$ simulations are obtained by fine-tuning the pre-trained lower velocity models ($U = 7 - 10\,\mathrm{m/s}$), and for the lower velocity case, $U_{ref} = 7\,\mathrm{m/s}$ simulations are obtained by fine-tuning the pre-trained higher velocity models ($U = 8 - 11\,\mathrm{m/s}$). Reference simulations are used to evaluate the quality of the simulation produced by transfer learning. We observe that on performing 2000 epochs of fine-tuning in this transfer learning task, the simulation agrees with the reference simulations. Also, for transfer learning, only one-fourth of the simulated measurement data points are required to reach convergence compared to the model trained from scratch. Figure 14 shows the streamwise velocity along the centerline at the hub height for the model, after transfer learning, compared to the reference (dashed lines) and pre-trained models (solid lines). The markers denote the simulation after transfer learning, with values indicating the reference velocity of the pre-trained model. To simulate the case with different velocity and turbulence intensity from a pretrained model with different conditions, we used models trained with $U = 7$ and11 m/s at a turbulence intensity of 10% as the pre-trained models. The target is to fine-tune these models to simulate a case with $U = 9\,\mathrm{m/s}$ at turbulence intensities of 6% and 8%. Similar to the first scenario, 2000 epochs of fine-tuning turned out to be effective. Figure 15 shows the streamwise velocity along the centerline at the hub height for the model, after transfer learning, compared to the reference (dashed lines) and pre-trained models (solid lines). A comparison of the velocity in a horizontal plane at hub height at different downstream locations for the two scenarios is shown in Appendix I. The results show good agreement between the reference and the simulations from the fine-tuned model. Fine-tuning took 274.60±1.43 min compared to 2087.15±9.90 min for the training from scratch. We see that the PINN model with transfer learning converged 7.5× faster compared to the model trained from scratch with lesser training data. While the transfer learning shows promising results for different inlet velocities and turbulence intensities, its performance under changes in turbine configuration currently requires substantially more fine-tuning and higher training epochs, which limits its practical effectiveness. Regardless of the fact that an end-to-end PINN training is data-hungry and computationally

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

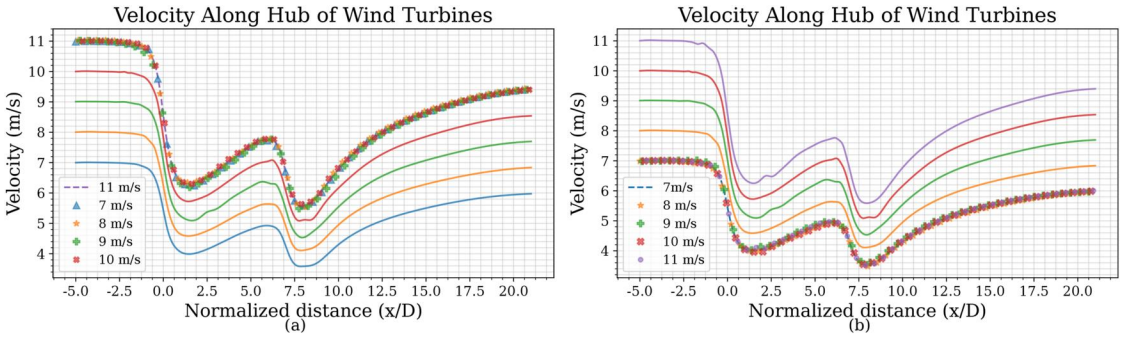



**FIG. 14.** Velocity along the streamwise centerline at hub height of the wind turbines simulated by the transfer learning strategy. (a) Higher velocity ($U_{ref} = 11\,\mathrm{m/s}$) simulation from pre-trained lower velocity ($U_{ref} = 7, 8, 9,$ and $10\,\mathrm{m/s}$) model, (b) lower velocity ($U_{ref} = 7\,\mathrm{m/s}$) simulation from pre-trained higher velocity ($U_{ref} = 8, 9, 10,$ and $11\,\mathrm{m/s}$) model. The dashed line is the target in each case, starting from the pre-trained model denoted by solid lines, whose velocity corresponds to the marker color. Markers denote the simulation after transfer learning, and the marker value is the $U_{ref}$ of the pre-trained model.

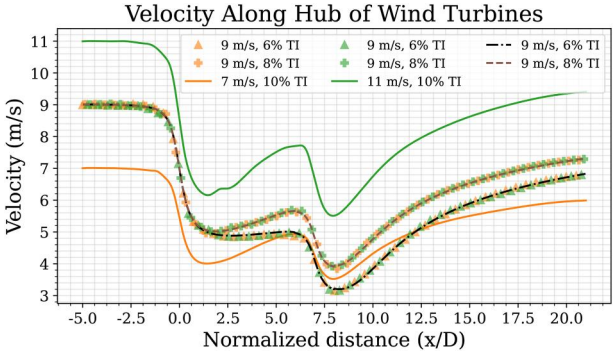



**FIG. 15.** Velocity along the streamwise centerline at hub height of the wind turbines simulated by the transfer learning strategy. $U = 9\,\mathrm{m/s}$ at turbulence intensities of 6% and 8% simulations from a pre-trained model with $U = 7$and$11\,\mathrm{m/s}$ at turbulence intensity of 10%. The dashed line is the target in each case, starting from the pre-trained model denoted by solid lines. Markers denote the simulation after transfer learning.

expensive, adopting approaches like transfer learning combined with the utility of PINN models to perform fast inferences makes them a viable option in real-world simulations even with sparse data availability. Although PINNs may fall short of the accuracy and efficiency of high-order CFD solvers for conventional forward problems and are therefore unlikely to fully supplant existing CFD, they nevertheless offer a realistic alternative when dealing with problems with partial information.

## V. CONCLUSION

A physics-informed neural network model using hybrid PDE and data-based training is developed to simulate turbulent wake on a grid of wind turbines using the RANS formulation with the 2-equation $k-\varepsilon$ turbulence closure model. The training strategy uses a mixture of PDE-residual losses, boundary condition losses, and measurement/simulation data losses. The simulations from the trained PINN model showed excellent consistency with the reference for different atmospheric inlet flow scenarios. The use of simulated observation data were only on a sparse grid for a single flow variable from the numerical solver. Nevertheless, the PINN model was able to predict all variables accurately for the entire domain. In addition, these data, which are confined to a single plane in the three-dimensional domain, can easily be replaced with sensory measurements, thus avoiding the need for a forward numerical solver during the training phase. A transfer learning strategy is developed to reduce the training time for PINN model in new atmospheric flow configurations when starting from a pre-trained PINN model with a different flow configuration. Our results demonstrate the possibility of using PINNs as a framework for simulating turbulent wake in multi-turbine settings.

Although the proposed framework does not currently exceed the computational efficiency of state-of-the-art spectral solvers, it provides a compelling proof of concept: turbulence in engineering application domain can be modeled using physics-informed neural networks, yielding continuous, mesh-free solutions unconstrained by conventional discretization. The present study employs a downsampled CFD dataset as a proxy for sparse measurements; future work should assess model performance under truly sparse, noisy real-world sensor data such as those obtainable from dual-Doppler LiDAR deployments. With future advances in wider generalizations and newer PINN architectures, our results will serve as a benchmark.

## ACKNOWLEDGMENTS

We thank all members of the QUEST Lab at IISc for useful discussions. The authors are grateful for the Prime Minister's Research Fellowship (PMRF) from the Ministry of Education to Azhar Gafoor CTP. This research is partially supported by research grants from the Ministry of Earth Sciences (MoES/36/OOIS/Extra/84/2022) and Shell India Markets Private Limited.

## AUTHOR DECLARATIONS

### Conflict of Interest

The authors have no conflicts to disclose.

### Author Contributions

**Azhar Gafoor CTP:** Conceptualization (equal); Data curation (equal); Formal analysis (equal); Investigation (equal); Methodology (equal); Software (equal); Validation (equal); Visualization (equal); Writing – original draft (equal). **Sumanth Kumar Boya:** Investigation (supporting); Methodology (supporting); Software (supporting); Visualization

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

(supporting). **Rishi Jinka:** Investigation (supporting); Methodology (supporting). **Abhineet Gupta:** Software (supporting); Validation (equal). **Deepak N. Subramani:** Conceptualization (equal); Formal analysis (equal); Methodology (equal); Project administration (equal); Supervision (equal); Validation (equal); Writing – review & editing (equal).

## DATA AVAILABILITY

The data that support the findings of this study are openly available in GitHub and Zenodo at https://github.com/quest-lab-iisc/PINNs_multi_turbine_wake_simulation, Ref. 71

## APPENDIX A: WEIGHTS FOR DIFFERENT LOSSES

Unlike conventional numerical methods that update parameters by sequentially iterating through discretized equations, Physics-Informed Neural Networks (PINNs) process all governing PDEs and other supervised components (boundary and data losses) simultaneously during training. This approach can lead to a single, dominant PDE overly influencing the network's learning, causing the effects of other equations to be ignored or boundary/data losses to overshadow residual losses, or vice versa, resulting in the model becoming stuck in some local minima. To resolve this imbalance, a weighting strategy is implemented to assign equal importance to all equations and boundary/data losses throughout the training process. The RMSEs of the normalized velocity and TKE at different locations in the vertical plane along the height of the hub for varying weights for different losses and equations are reported in Table V. It can be observed how varying the weights affects the model performance. On weighing the coefficients $\alpha_b$ and $\alpha_d$ 10× more compared to the residual coefficient, $\alpha_r$, and giving weights in the ratio 100 : 10 : 1 : 1 for the coefficients $\alpha_{1-4}$, the PINN model showed overall better accuracy.

## APPENDIX B: TRAINING LOSS CONVERGENCE

Figure 16 shows the evolution of the individual loss components during training, including the boundary/data loss, total PDE residual, continuity equation, three-momentum equations, and the

**TABLE V.** RMSE between the simulation from the PINN model and the numerical solver for varying weights for different losses and equations. Normalized velocity and TKE are compared at the inlet, 3D, 6D, 9D, 12D, and 15D downstream of the turbine in two vertical planes through the center of the wind turbines at $\pm 3.5D$. Bold and underlined values denote the lowest RMSE in each row.

| | Boundary/Data | $\alpha_b$, $\alpha_d$ | 1 | 1 | 1 | 10 | 10 |
|---|---|---|---|---|---|---|---|
| | Residual | $\alpha_r$ | 1 | 1 | 1 | 1 | 1 |
| | Continuity equation | $\alpha_1$ | 1 | 10 | 100 | 100 | 100 |
| | Momentum equation | $\alpha_2$ | 1 | 1 | 10 | 10 | 10 |
| | k-transport equation | $\alpha_3$ | 1 | 1 | 1 | 1 | 10 |
| | $\varepsilon$-transport equation | $\alpha_4$ | 1 | 1 | 1 | 1 | 1 |
| Inlet | Normalized velocity | **−3.5** | 0.030 14 | 0.030 24 | 0.052 61 | **0.029 38** | 0.037 34 |
| | | **3.5** | 0.029 90 | 0.034 02 | 0.052 22 | **0.029 24** | 0.037 77 |
| | Normalized TKE | **−3.5** | 0.037 14 | 0.114 16 | 0.026 48 | 0.029 40 | **0.022 95** |
| | | **3.5** | 0.033 50 | 0.146 41 | 0.037 12 | 0.033 43 | **0.030 91** |
| 3D | Normalized velocity | **−3.5** | 0.036 54 | 0.041 91 | 0.047 21 | **0.030 30** | 0.054 40 |
| | | **3.5** | 0.039 81 | 0.039 61 | 0.042 09 | **0.034 16** | 0.05374 |
| | Normalized TKE | **−3.5** | 0.667 95 | 0.697 25 | 0.425 17 | **0.381 78** | 0.674 81 |
| | | **3.5** | 0.606 67 | 0.634 88 | 0.417 22 | **0.365 82** | 0.714 69 |
| 6D | Normalized velocity | **−3.5** | 0.034 11 | 0.034 02 | 0.048 73 | **0.030 68** | 0.049 11 |
| | | **3.5** | 0.036 46 | 0.040 86 | 0.038 72 | **0.035 33** | 0.056 63 |
| | Normalized TKE | **−3.5** | 0.654 82 | 0.683 76 | 0.416 33 | **0.313 67** | 0.631 33 |
| | | **3.5** | 0.639 76 | 0.657 93 | 0.376 99 | **0.304 61** | 0.695 91 |
| 9D | Normalized velocity | **−3.5** | 0.043 14 | 0.046 33 | 0.055 66 | **0.032 72** | 0.052 22 |
| | | **3.5** | 0.056 11 | 0.048 11 | 0.046 62 | **0.039 49** | 0.061 43 |
| | Normalized TKE | **−3.5** | 0.713 57 | 0.769 35 | 0.667 66 | **0.473 68** | 0.744 08 |
| | | **3.5** | 0.737 91 | 0.736 38 | 0.695 78 | **0.498 94** | 0.776 38 |
| 12D | Normalized velocity | **−3.5** | **0.034 71** | 0.037 62 | 0.052 41 | 0.036 27 | 0.053 15 |
| | | **3.5** | **0.037 58** | 0.039 31 | 0.044 15 | 0.041 01 | 0.058 90 |
| | Normalized TKE | **−3.5** | 0.644 50 | 0.657 95 | 0.545 98 | **0.453 51** | 0.621 71 |
| | | **3.5** | 0.639 42 | 0.603 22 | 0.527 95 | **0.448 19** | 0.620 52 |
| 15D | Normalized velocity | **−3.5** | **0.037 98** | 0.0256 | 0.053 08 | 0.041 58 | 0.055 64 |
| | | **3.5** | **0.031 17** | 0.035 94 | 0.047 50 | 0.045 72 | 0.060 09 |
| | Normalized TKE | **−3.5** | 0.558 96 | 0.534 99 | 0.504 29 | **0.449 09** | 0.517 31 |
| | | **3.5** | 0.545 74 | 0.502 17 | 0.493 06 | **0.437 06** | 0.506 16 |

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

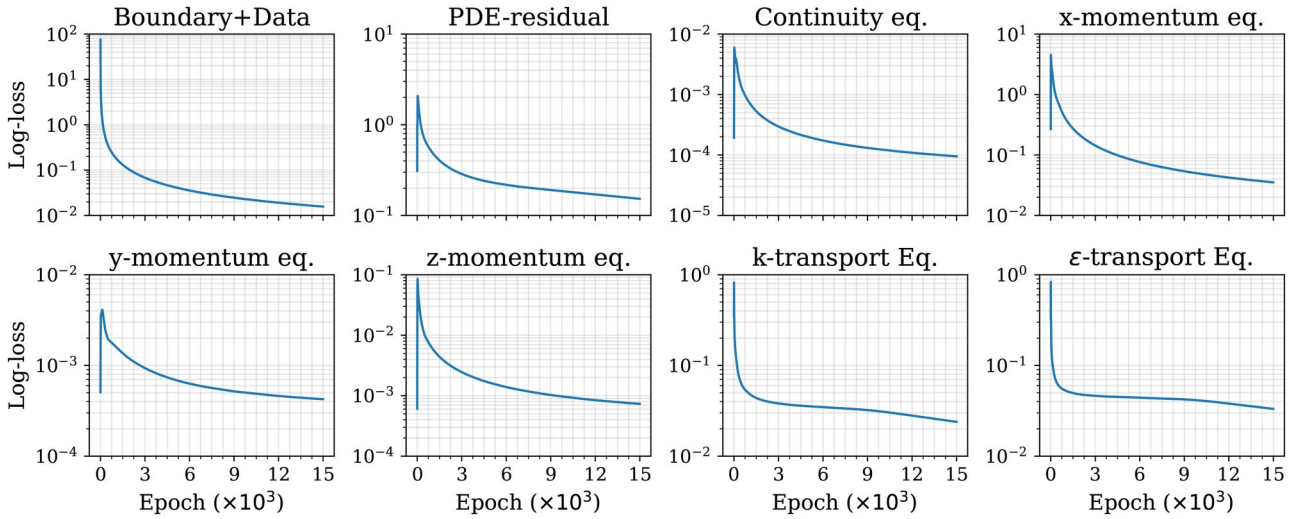



**FIG. 16.** Convergence of the individual loss components during training, including boundary/data loss, PDE residual, continuity, momentum, k-transport, and $\varepsilon$-transport equations.

transport equations for turbulent kinetic energy and dissipation rate. All components exhibit a consistent monotonic decrease over the course of training, indicating stable optimization and balanced enforcement of the governing equations and boundary conditions.

## APPENDIX C: ABLATIONS

The ablation studies are carried out for simulations in a domain with a rectangular grid of turbines, with $U_{ref} = 10\,\mathrm{m/s}$ and a turbulence intensity of 10%.

### 1. Model parameters

Table VI lists the RMSE between the simulation from the PINN model and the numerical solver for different numbers of model parameters, along with the training time. The parameters are varied by changing the number of neurons in the three common hidden layers from 600 to 800. The model with 3 090 200 parameters is chosen for subsequent experiments because it offers improved performance over smaller models while providing a practical balance between computational cost and accuracy compared with larger models.

### 2. Number of collocation points

The collocation points for the three-dimensional domain are shown in Figs. 4 and 19. The blue region represents the entire domain, with a higher density near the actuator disk region, denoted by the green region. The PINN training accuracy depends on the number of sampled points where the losses are calculated. The total number of collocation points depends on the base size ($b$). The sampling is done sequentially: first, the entire domain is sampled with $b \times 10^4$ points, then a refined region is sampled with $b \times 10^3$ points around the turbine center, and finally, $b \times 10^3$ points are sampled for the actuator disk region. During each sampling stage, points sampled in the former stage in the overlapping regions are omitted. Thus, the entire collocation sample points will be less than $b \times (10^4 + 10^3 + 10^3)$. Table VII lists the RMSE between the simulation from the PINN model and the numerical solver for different numbers of collocation points. A significant gain in accuracy is observed when the base size is increased from 2 to 3. Even though the mean RMSE value reduces further on increasing the base size to 4, it comes at a higher computational cost, especially when the model needs to be trained for higher epochs. Therefore, a base size of 3 is chosen as the optimal as it offers a good balance between computational cost and accuracy.

### 3. Number of data points

Table VIII summarizes RMSE between the PINN solution and the numerical solver for different numbers of sampled data points and sampling strategies. The simulation was initiated with a higher number of sampled points, and the progression of error with the

**TABLE VI.** RMSE between the simulation from the PINN model and the numerical solver for different numbers of model parameters. Normalized velocity is compared at the inlet, 3D, 6D, 9D, 12D, and 15D downstream of the turbine in a horizontal plane at the hub height of the wind turbines. The mean of RMSE is also listed.

| Parameters | Time/1000 epochs (mins) | Inlet | 3D | 6D | 9D | 12D | 15D | Mean |
|---|---|---|---|---|---|---|---|---|
| 3 690 800 | 150.81 | 0.001 31 | 0.001 08 | 0.001 03 | 0.000 79 | 0.000 44 | 0.000 56 | 0.000 87 |
| 3 090 200 | 139.14 | 0.001 19 | 0.001 24 | 0.001 20 | 0.000 79 | 0.000 51 | 0.000 59 | 0.000 92 |
| 2 529 600 | 116.09 | 0.000 93 | 0.001 73 | 0.001 67 | 0.002 28 | 0.001 35 | 0.001 08 | 0.001 51 |

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

**TABLE VII.** RMSE between the simulation from the PINN model and the numerical solver for different numbers of collocation points. Normalized velocity is compared at the inlet, 3D, 6D, 9D, 12D, and 15D downstream of the turbine in a horizontal plane at the hub height of the wind turbines. The mean of RMSE is also listed. Higher base size denotes higher points.

| Base size | Time/1000 epochs (mins) | Inlet | 3D | 6D | 9D | 12D | 15D | Mean |
|---|---|---|---|---|---|---|---|---|
| 4 | 156.53 | 0.001 28 | 0.001 02 | 0.000 98 | 0.000 80 | 0.000 44 | 0.000 50 | 0.000 84 |
| 3 | 139.14 | 0.001 19 | 0.001 24 | 0.001 20 | 0.000 79 | 0.000 51 | 0.000 59 | 0.000 92 |
| 2 | 123.89 | 0.001 08 | 0.002 02 | 0.002 15 | 0.002 79 | 0.001 28 | 0.000 86 | 0.001 70 |

**TABLE VIII.** RMSE between the simulation from the PINN model and the numerical solver for different numbers of sampled data points and sampling strategies. Normalized velocity is compared at the inlet, 3D, 6D, 9D, 12D, and 15D downstream of the turbine in a horizontal plane at the hub height of the wind turbines. The mean of RMSE is also listed.

| Method | No. of data points | Inlet | 3D | 6D | 9D | 12D | 15D | Mean |
|---|---|---|---|---|---|---|---|---|
| Random | 8000 | 0.000 97 | 0.001 15 | 0.000 96 | 0.000 84 | 0.000 47 | 0.000 52 | 0.000 82 |
| | 4000 | 0.001 52 | 0.001 17 | 0.001 11 | 0.000 77 | 0.000 49 | 0.000 55 | 0.000 94 |
| | 2000 | 0.001 07 | 0.002 72 | 0.002 95 | 0.003 49 | 0.001 53 | 0.001 01 | 0.002 13 |
| Grid-based | 3876 | 0.001 19 | 0.001 24 | 0.001 20 | 0.000 79 | 0.000 51 | 0.000 59 | 0.000 92 |
| | 1938 | 0.001 33 | 0.002 22 | 0.003 05 | 0.003 34 | 0.001 29 | 0.000 89 | 0.002 02 |
| Random % | 3873 ($\approx$2%) | 0.001 13 | 0.001 11 | 0.000 90 | 0.001 49 | 0.000 45 | 0.000 49 | 0.000 93 |
| | 1923 ($\approx$1%) | 0.001 37 | 0.002 87 | 0.003 19 | 0.003 38 | 0.001 54 | 0.000 96 | 0.002 22 |

reduction of the number of data points was observed. The mean RMSE of the simulation with 8000 randomly sampled data points is 0.000 82. Reducing the number by 50% to 4000 yields a mean RMSE of 0.000 94, which is approximately only 15% worse. Upon further reducing the number of sampled points to 2000, a significant increase ($\sim$126%) in the mean RMSE value is observed. A similar trend in RMSE is observed when a different sampling strategy, grid-based, is used. In this method, a structured coarse grid of data points is sampled from the fine mesh-grid by skipping intermediate grid points in an orderly fashion (see Appendix B). A slightly better mean RMSE (0.000 92) is obtained using the grid-based approach (3876 data points) compared to random sampling. Furthermore, instead of randomly sampling from the entire available data, a more targeted approach is used to ensure that the sampled dataset maintains a uniform distribution of points across the domain. In this approach, a fixed percentage of data points is randomly sampled, ensuring spatial representativeness in both the streamwise and spanwise directions. This method also substantiates the findings of the previous approaches, yielding similar mean RMSE values for a similar sample size. Based on this evidence, we adopt $\approx$4000 sampling points for subsequent experiments as a practical compromise between computational cost and predictive fidelity. The horizontal plane of the computational domain spans 26D $\times$ 17D, corresponding to an area of 400 456.42 m$^2$ for D = 30.1 m. While the CFD solver employs 193 776 grid points to resolve this plane, the PINN model achieves comparable accuracy using only 2% of that number. That is one sample point in approximately 100 m$^2$.

## 4. Effect of wall functions

To motivate the inclusion of wall functions in the PINN loss formulation, we conduct an ablation study based on a two-dimensional turbulent channel flow benchmark. This channel flow configuration is a standard validation problem for turbulent flow solvers, with DNS reference data available over a range of Reynolds numbers.[96]

### a. Problem setup

The flow between two flat plates separated by a distance of $2h$ is driven by a constant pressure gradient in the streamwise direction. The simulation is performed on half the channel height, exploiting the symmetry of the fully developed flow (Fig. 17). The friction Reynolds number is defined as $Re_\tau = u_\tau h/\nu$, and the study is conducted at $Re_\tau = 180$. Setting $u_\tau = 1$ m/s and $h = 1$ m, the kinematic viscosity is $\nu = 1/180$ m$^2$/s, and the constant pressure gradient is $\partial p/\partial x = -1$ Pa/m for density $\rho = 1$ kg/m$^3$.

The PINN model solves the RANS equations with the $k$–$\varepsilon$ turbulence closure (Two-dimensional implementation of Eq. (1)

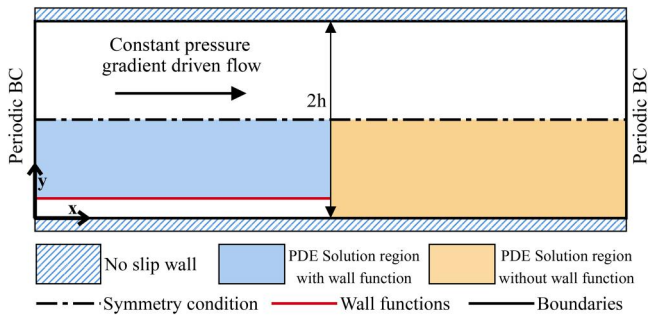



**FIG. 17.** Schematic of the domain for turbulent channel flow together with the boundary conditions.

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

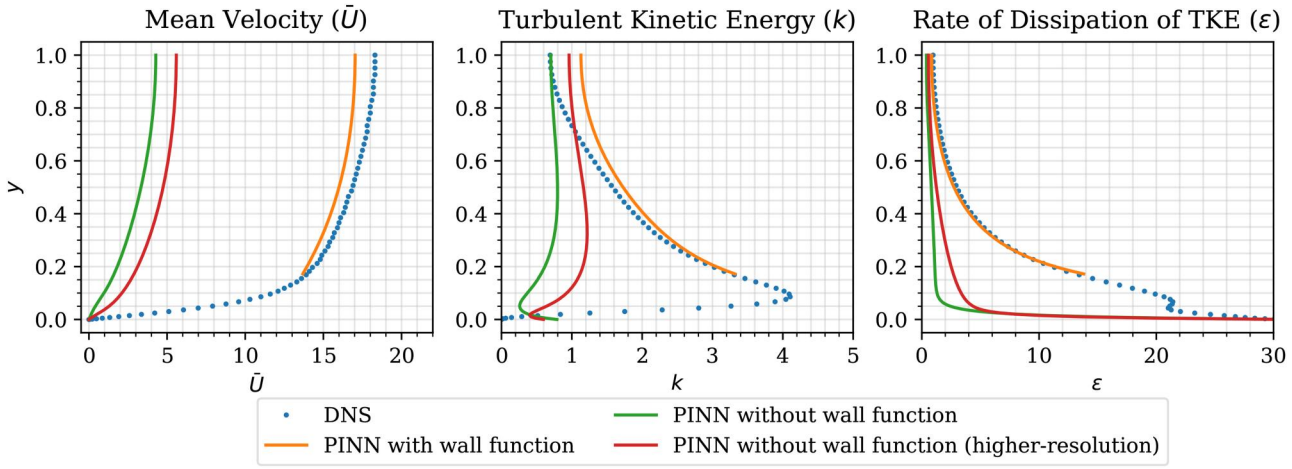



**FIG. 18.** Comparison of mean velocity ($\bar{U}$), turbulent kinetic energy ($k$), and rate of dissipation of TKE ($\varepsilon$) profiles at $Re_\tau = 180$ for three PINN configurations against DNS data:[96] PINN with wall function (orange), PINN without wall function (green), and PINN without wall function with refinement (red).

without source terms). Periodic boundary conditions are applied at the inlet and outlet, and symmetry conditions are imposed at the top boundary. Two configurations are compared: (i) a baseline model, in which no wall function constraints are applied, and the bottom boundary is placed at $y^+ = 0$, and (ii) a wall-function model, in which the bottom boundary is located at $y^+ = 31$ (within the log-law region, $30 < y^+ < 500$) and the following wall function expressions are enforced as boundary conditions:

$$u = \frac{u_\tau}{\kappa}\ln y^+ + B, \qquad k = \frac{u_\tau^2}{\sqrt{C_\mu}}, \qquad \varepsilon = \frac{u_\tau^3}{\kappa y}. \tag{C1}$$

All other hyperparameters, network architecture, and training settings are held identical between the two configurations.

### b. Results

Figure 18 presents a comparison of the predicted profiles of $\bar{U}$, $k$, and $\varepsilon$ with the DNS reference data of Moser, Kim, and Mansour[96] for the three PINN configurations at $Re_\tau = 180$. The PINN employing wall functions (orange) reproduces the DNS profiles with good accuracy across all three variables. The mean velocity profile exhibits the expected logarithmic behavior and attains the correct centerline value, while the $k$ and $\varepsilon$ distributions remain in close agreement with the DNS data over the entire channel height.

In contrast, neither of the PINN setups without wall functions is able to reproduce the correct solution. The baseline model (green), which uses the same number of collocation points as the wall-function model, substantially underestimates the centerline velocity and produces erroneous near-wall behavior in $k$ and $\varepsilon$. This suggests that the optimizer has converged to an incorrect local minimum in the loss landscape. Even increasing the number of collocation points by an order of 10 relative to the baseline (red) fails to resolve the issue: this higher-resolution model without wall functions still deviates markedly from the DNS data. This outcome indicates that the optimization challenge cannot be attributed solely to spatial resolution, but instead arises fundamentally from the lack of physically consistent near-wall boundary conditions.

These findings demonstrate that wall functions are crucial, as they impose consistent boundary expressions for $\bar{U}$, $k$, and $\varepsilon$, thereby restricting the admissible solution space and steering the PINN optimizer toward physically accurate solutions.

## APPENDIX D: IMPLEMENTATION DETAILS

### 1. Domain sampling

Figure 19 shows a three-dimensional schematic of the computational domain illustrating the sampling points used for the

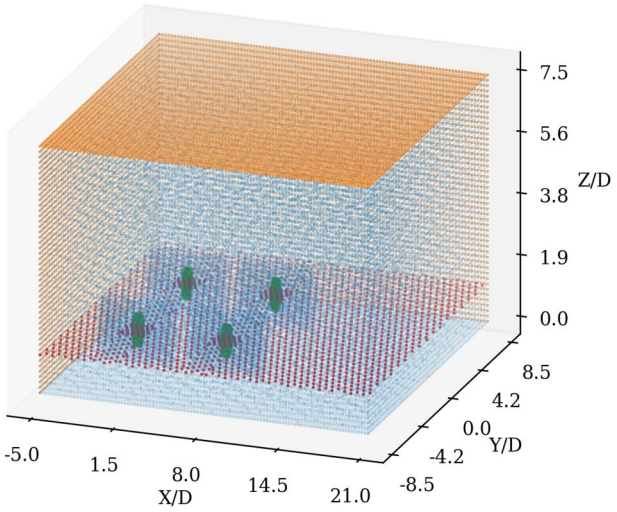



**FIG. 19.** Three-dimensional schematic of the domain sampling points showing the points used for different loss calculations. (i) Orange: boundary points (inlet, lateral rear, and top boundaries are shown), (ii) red: data points in a horizontal plane through the hub of the turbines, (iii) blue: domain collocation points, and (iv) green: actuator disk region.

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

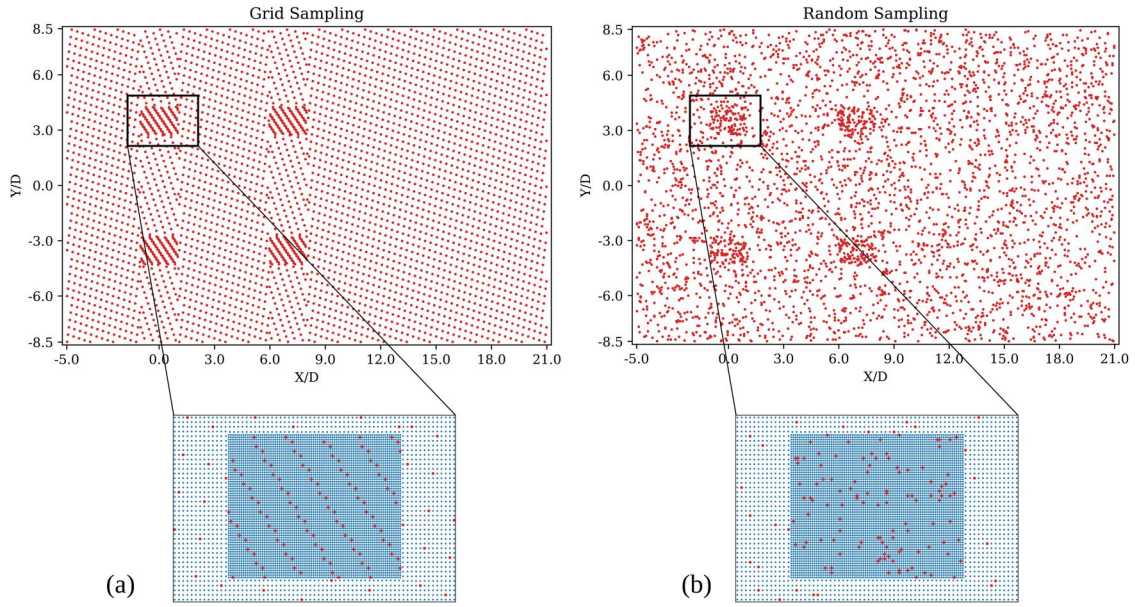



**FIG. 20.** Two different sampling strategies for the data points in a horizontal plane through the hub of the turbines (a) grid-based and (b) random sampling. The bottom row shows the sampling near the actuator disk region. The dense CFD mesh points are shown in blue.

different loss calculations during the training of the PINN model. The domain consists of a rectangular volume enclosing the wind turbine grid, with distributed sampling points for boundary, data, and PDE-residual losses. The orange points represent the domain boundaries (Inlet, lateral rear, and top planes are shown in the figure). These points are used to impose boundary conditions that govern inflow velocity profiles, wall conditions, lateral confinement, and outflow behavior. The red points define a horizontal sampling plane intersecting the hub height of all turbines. The streamwise velocity corresponding to the red points is used in the hybrid training of the PINN model. The blue and green points correspond to the domain collocation points, where the governing equations are solved to evaluate the PDE-residual losses. A noticeable refinement in point density is applied near the turbine regions to better capture the strong velocity gradients and wake recovery. The green regions indicate the actuator disk zones, which model the turbines as momentum sinks using the source terms in the governing equations.

### 2. Data sampling

Figure 20 shows two strategies used for data points sampling in the horizontal plane passing through the hub height of the turbines: (i) grid-based and (ii) random sampling. In both cases, the sampling points (shown in red) are overlaid on the computational mesh (shown in blue), highlighting the underlying CFD resolution. In the grid-based sampling approach [Fig. 20(a)], data points are uniformly distributed over the plane with a constant spacing, and the random sampling approach [Fig. 20(b)] introduces stochastic variation in the spatial distribution of points.

## APPENDIX E: EVALUATION ACROSS RANDOM INITIALIZATIONS

The PINN model experiment was repeated with multiple random seeds (N = 5) to account for stochastic variations in initialization, data shuffling, and optimization. The domain with a rectangular grid of turbines for $U_{ref} = 9\,\mathrm{m/s}$ at a turbulence intensity of 10% is simulated over multiple runs with different random seeds. The performance metrics reported in Tables IX and X represent the mean ± standard deviation computed in these independent runs for the RMSE between the simulation from the PINN model and the numerical solver. The results indicate consistent convergence behavior and limited dependence on initialization. This consistency supports the reproducibility and reliability of the proposed approach across independent trials.

## APPENDIX F: ERROR MAP

Figure 21 shows the comparison of velocity and turbulence intensity between the PINN model and the CFD solver in a vertical plane, along with the absolute error map.

## APPENDIX G: DIAMOND GRID: SIMULATION

Similar results shown for the rectangular grid of turbines in Sec. III D are shown here for another pattern, the diamond grid (Fig. 22). Figure 23 shows the contour plot of the RANS-PINN turbulent flow simulations (velocity, turbulence intensity, and pressure) in horizontal and two vertical planes ($y/D = \pm 3.5$ and 0),

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

**TABLE IX.** RMSE between the simulation from the PINN model and the numerical solver over multiple runs with different random seeds. Normalized velocity and TKE are compared at the inlet, 3D, 6D, 9D, 12D, and 15D downstream of the turbine in a horizontal plane at the hub height of the wind turbines.

| | | Run 1 | Run 2 | Run 3 | Run 4 | Run 5 | Mean |
|---|---|---|---|---|---|---|---|
| Inlet | Normalized velocity | 0.001 44 | 0.001 03 | 0.000 77 | 0.001 67 | 0.001 05 | 0.001 19 ± 0.00032 |
| | Normalized TKE | 0.005 06 | 0.003 87 | 0.003 36 | 0.007 44 | 0.003 93 | 0.004 73 ± 0.001 46 |
| 3D | Normalized velocity | 0.002 47 | 0.001 81 | 0.002 88 | 0.001 95 | 0.001 76 | 0.002 17 ± 0.000 43 |
| | Normalized TKE | 0.313 59 | 0.320 14 | 0.314 41 | 0.452 91 | 0.545 26 | 0.389 26 ± 0.094 33 |
| 6D | Normalized velocity | 0.001 99 | 0.002 02 | 0.002 28 | 0.002 29 | 0.001 77 | 0.002 07 ± 0.000 20 |
| | Normalized TKE | 0.380 19 | 0.384 83 | 0.352 12 | 0.354 49 | 0.542 59 | 0.402 84 ± 0.071 10 |
| 9D | Normalized velocity | 0.002 83 | 0.002 45 | 0.002 62 | 0.001 51 | 0.002 58 | 0.002 40 ± 0.000 46 |
| | Normalized TKE | 0.576 98 | 0.412 27 | 0.382 58 | 0.357 36 | 0.677 90 | 0.481 42 ± 0.124 64 |
| 12D | Normalized velocity | 0.002 29 | 0.002 25 | 0.002 37 | 0.001 21 | 0.002 29 | 0.002 08 ± 0.000 44 |
| | Normalized TKE | 0.649 46 | 0.470 87 | 0.414 70 | 0.376 87 | 0.598 15 | 0.502 01 ± 0.105 11 |
| 15D | Normalized velocity | 0.001 42 | 0.002 22 | 0.002 28 | 0.000 91 | 0.002 29 | 0.001 82 ± 0.000 56 |
| | Normalized TKE | 0.666 13 | 0.322 90 | 0.277 75 | 0.344 03 | 0.513 20 | 0.424 80 ± 0.144 63 |

**TABLE X.** RMSE between the simulation from the PINN model and the numerical solver over multiple runs with different random seeds. Normalized velocity and TKE are compared at the inlet, 3D, 6D, 9D, 12D, and 15D downstream of the turbine in two vertical planes through the center of the wind turbines at ±3.5*D*.

| | | y/D | Run 1 | Run 2 | Run 3 | Run 4 | Run 5 | Mean |
|---|---|---|---|---|---|---|---|---|
| Inlet | Normalized velocity | −3.5 | 0.037 48 | 0.029 28 | 0.029 38 | 0.036 01 | 0.029 19 | 0.032 27 ± 0.00 369 |
| | | 3.5 | 0.030 25 | 0.029 24 | 0.029 24 | 0.055 35 | 0.029 10 | 0.034 64 ± 0.010 37 |
| | Normalized TKE | −3.5 | 0.018 12 | 0.032 11 | 0.029 40 | 0.018 42 | 0.028 54 | 0.025 32 ± 0.005 87 |
| | | 3.5 | 0.019 66 | 0.035 98 | 0.033 43 | 0.024 80 | 0.031 06 | 0.028 99 ± 0.005 96 |
| 3D | Normalized velocity | −3.5 | 0.052 69 | 0.031 21 | 0.030 30 | 0.070 82 | 0.035 11 | 0.044 03 ± 0.015 65 |
| | | 3.5 | 0.072 18 | 0.034 39 | 0.034 16 | 0.060 63 | 0.038 78 | 0.048 03 ± 0.015 53 |
| | Normalized TKE | −3.5 | 0.312 27 | 0.420 69 | 0.381 78 | 0.308 00 | 0.373 21 | 0.359 19 ± 0.043 15 |
| | | 3.5 | 0.445 58 | 0.388 78 | 0.365 82 | 0.399 01 | 0.435 57 | 0.406 95 ± 0.029 65 |
| 6D | Normalized velocity | −3.5 | 0.065 48 | 0.031 69 | 0.030 68 | 0.061 35 | 0.036 44 | 0.045 13 ± 0.015 11 |
| | | 3.5 | 0.068 93 | 0.035 92 | 0.035 33 | 0.063 00 | 0.038 87 | 0.048 41 ± 0.014 51 |
| | Normalized TKE | −3.5 | 0.343 62 | 0.346 23 | 0.313 67 | 0.314 89 | 0.328 40 | 0.329 36 ± 0.013 74 |
| | | 3.5 | 0.432 21 | 0.322 48 | 0.304 61 | 0.307 46 | 0.381 72 | 0.3497 ± 0.049 78 |
| 9D | Normalized velocity | −3.5 | 0.07455 | 0.032 74 | 0.032 72 | 0.061 52 | 0.035 47 | 0.0474 ± 0.017 37 |
| | | 3.5 | 0.051 32 | 0.039 24 | 0.039 49 | 0.056 14 | 0.038 14 | 0.044 87 ± 0.007 41 |
| | Normalized TKE | −3.5 | 0.610 56 | 0.484 89 | 0.473 68 | 0.473 71 | 0.456 92 | 0.499 95 ± 0.056 02 |
| | | 3.5 | 0.601 85 | 0.509 14 | 0.498 94 | 0.454 94 | 0.487 15 | 0.5104 ± 0.049 22 |
| 12D | Normalized velocity | −3.5 | 0.078 70 | 0.036 69 | 0.036 27 | 0.065 88 | 0.038 72 | 0.05125 ± 0.01767 |
| | | 3.5 | 0.058 85 | 0.040 98 | 0.041 01 | 0.066 20 | 0.040 75 | 0.049 56 ± 0.010 84 |
| | Normalized TKE | −3.5 | 0.631 41 | 0.464 99 | 0.453 51 | 0.575 40 | 0.391 90 | 0.503 44 ± 0.087 14 |
| | | 3.5 | 0.590 35 | 0.455 71 | 0.448 19 | 0.552 04 | 0.390 08 | 0.487 27 ± 0.0732 |
| 15D | Normalized velocity | −3.5 | 0.075 31 | 0.041 99 | 0.041 58 | 0.075 32 | 0.043 03 | 0.055 45 ± 0.016 23 |
| | | 3.5 | 0.068 15 | 0.045 67 | 0.045 72 | 0.075 05 | 0.045 12 | 0.055 94 ± 0.012 97 |
| | Normalized TKE | −3.5 | 0.576 24 | 0.463 28 | 0.449 09 | 0.555 40 | 0.356 11 | 0.480 02 ± 0.079 41 |
| | | 3.5 | 0.559 75 | 0.445 75 | 0.437 06 | 0.520 64 | 0.339 02 | 0.460 44 ± 0.076 15 |

passing through the hub of the wind turbines for the diamond grid. Figures 24 and 25 show the velocity and turbulent kinetic energy profiles in the horizontal and vertical planes, respectively, at different downstream locations, along with the simulation from the CFD k–$\omega$ SST solver for comparison. The simulations are shown for $U_{ref} = 10\,\mathrm{m/s}$.

## APPENDIX H: RECTANGULAR GRID: SIMULATIONS

### 1. Reference velocity: 8 m/s

Figure 26 shows the contour plot of the RANS-PINN turbulent flow simulations (velocity, turbulence intensity, and pressure) in horizontal and vertical planes passing through the hub of the

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

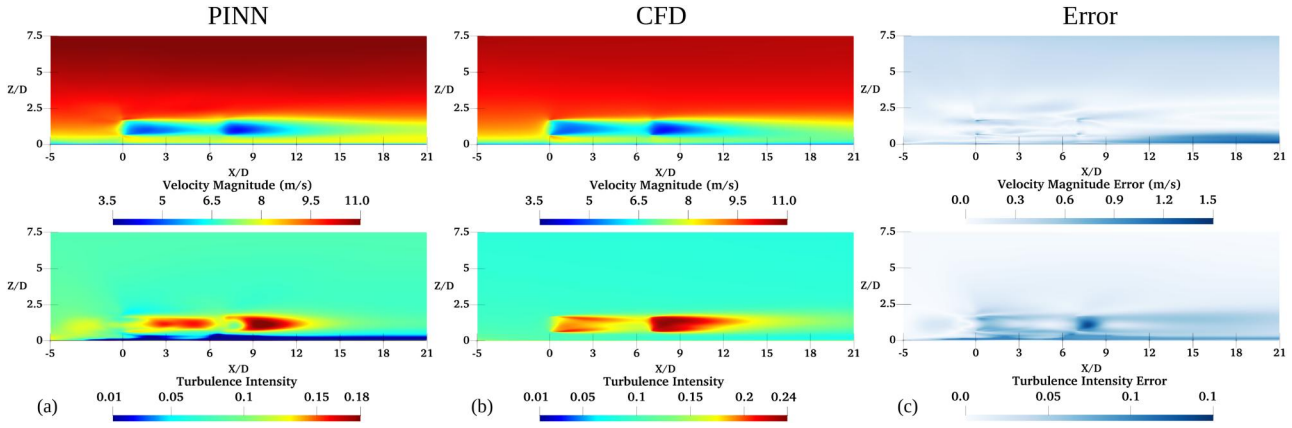



**FIG. 21.** Contours showing comparison of velocity and turbulence intensity between PINN model and CFD solver in a vertical plane through the center of wind turbines in a rectangular grid for $U_{ref} = 9\,\mathrm{m/s}$. (a) PINN, (b) CFD, and (c) error map: (i) top: velocity magnitude and (ii) bottom: turbulence intensity.

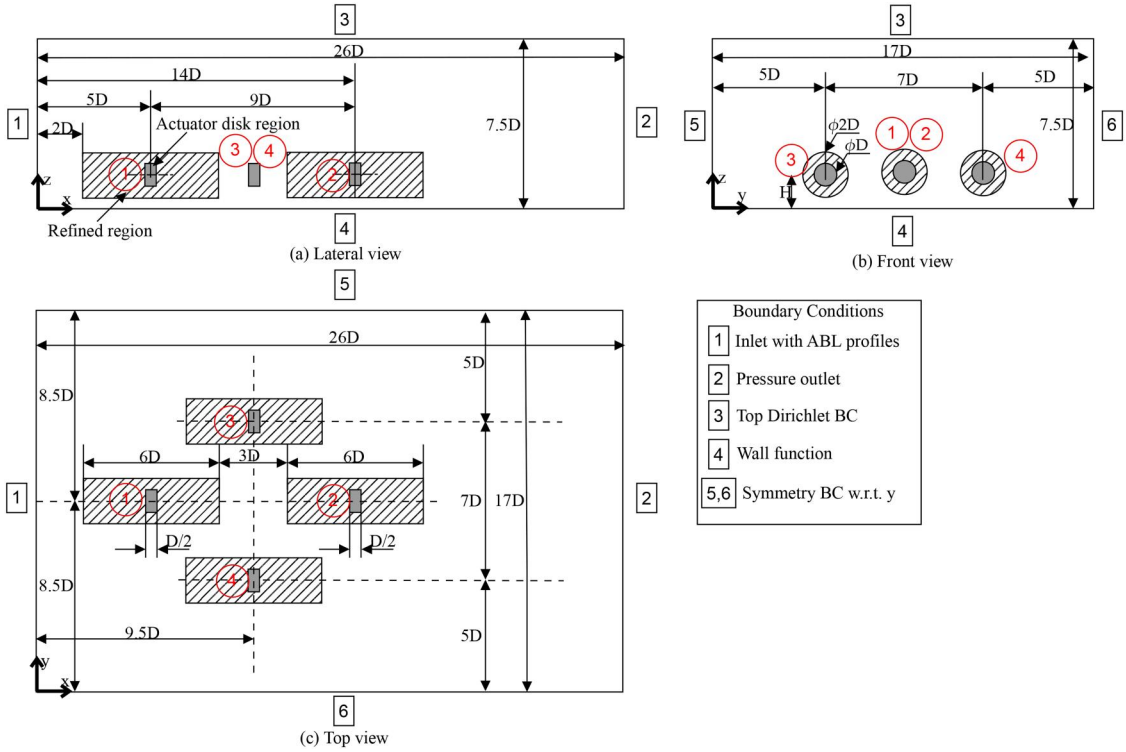



**FIG. 22.** Schematic of the diamond grid domain of wind turbines: (a) lateral view, (b) front view, and (c) top view. D is the diameter of the turbine, and H is the hub height. The boundary conditions used on the domain surfaces are also shown.

wind turbines for the rectangular grid. Figure 27 shows the velocity and turbulent kinetic energy profiles in the two planes at different downstream locations, along with the simulation from the CFD k–$\omega$ SST solver for comparison.

### 2. Reference velocity: 10 m/s

Figure 28 shows the contour plot of the RANS-PINN turbulent flow simulations (velocity, turbulence intensity, and pressure) in horizontal and vertical planes passing through the hub of the

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

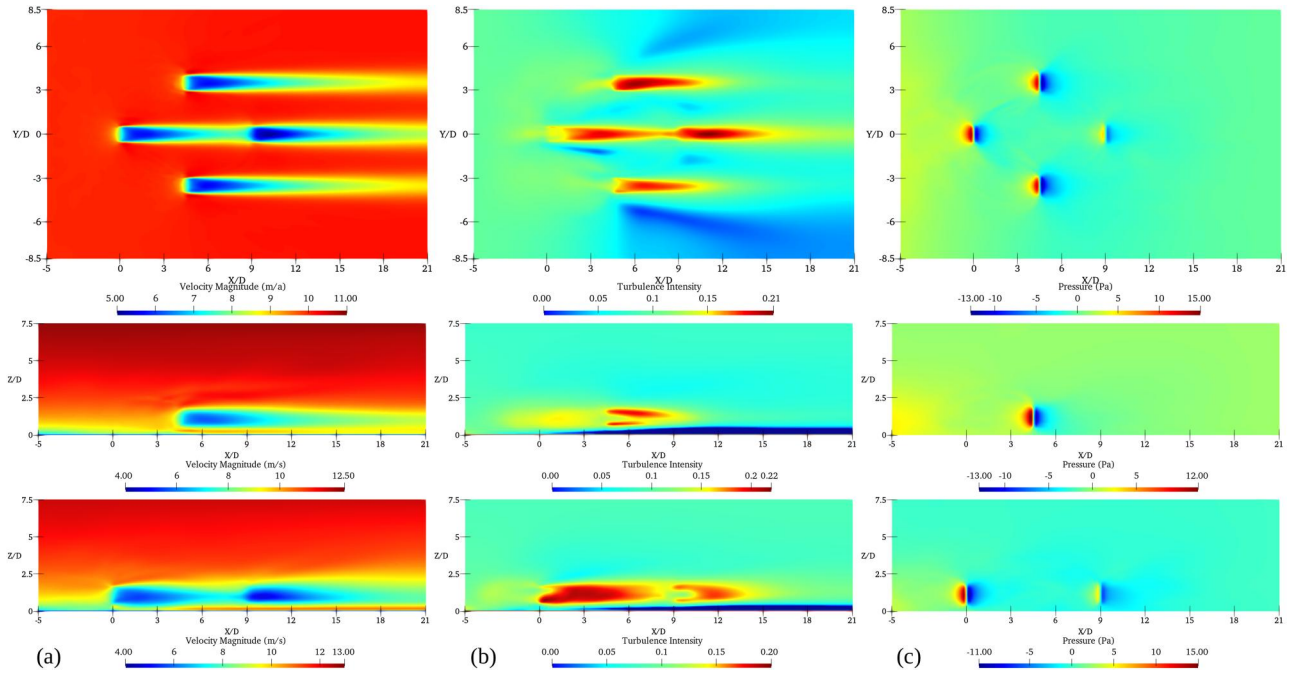



**FIG. 23.** Contours of (a) velocity, (b) turbulence intensity, and (c) pressure in three planes through the center of wind turbines: (i) top: horizontal plane, (ii) middle: vertical plane at y/D = ±3.5, and (iii) bottom: vertical plane at y/D = 0.

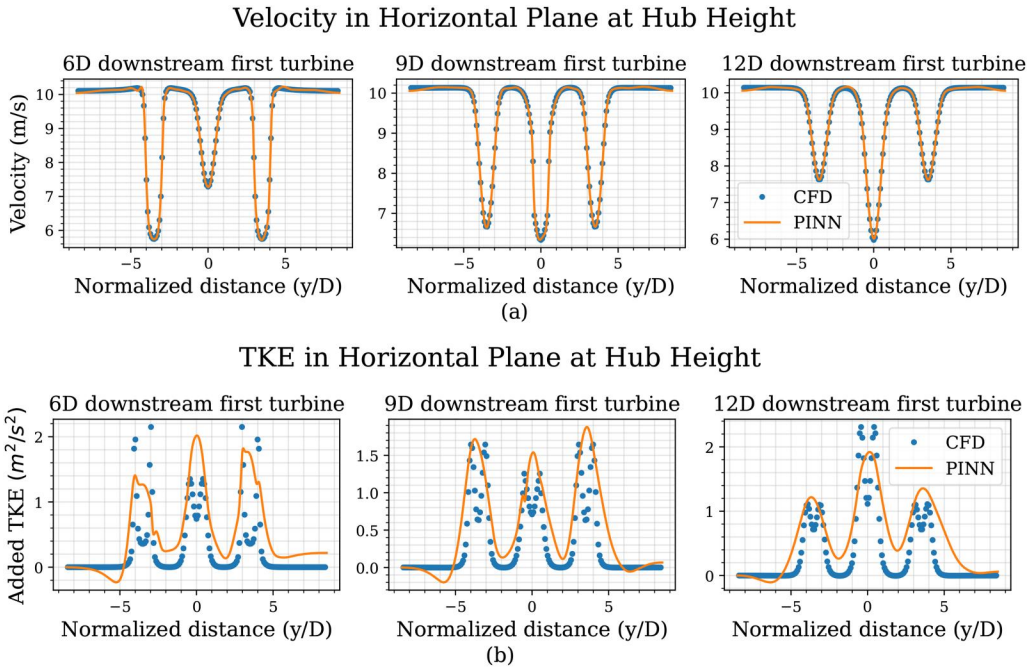



**FIG. 24.** Comparison of (a) velocity and (b) turbulent kinetic energy predicted by the PINN model with the numerical solver in a horizontal plane at the hub height of the wind turbines.

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

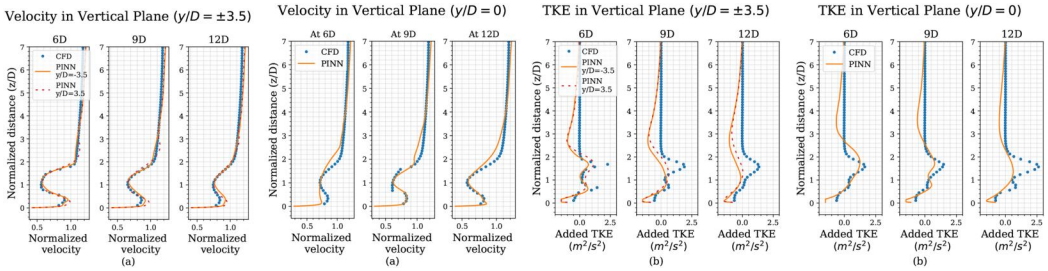



**FIG. 25.** Comparison of (a) velocity and (b) turbulent kinetic energy predicted by the PINN model with the numerical solver in a vertical plane through the center of wind turbines at y/D = ±3.5 and 0.

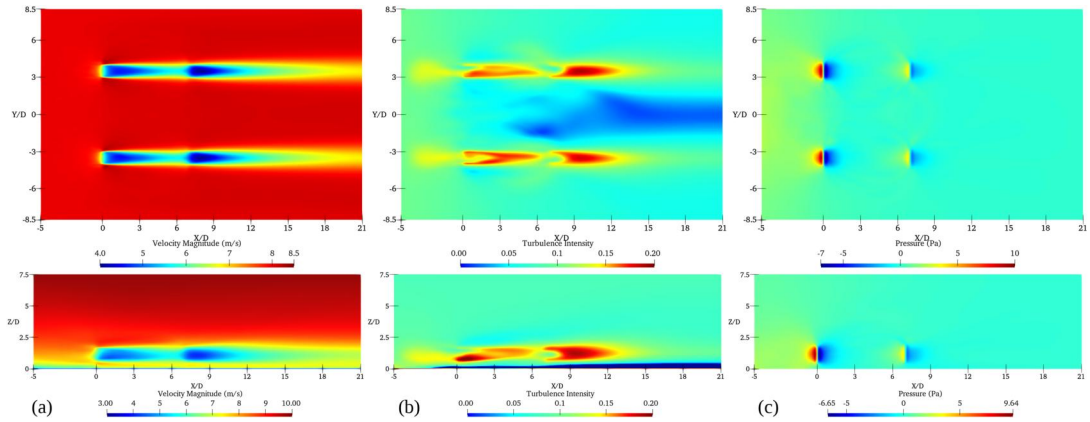



**FIG. 26.** Contours of (a) velocity, (b) turbulence intensity, and (c) pressure in two planes through the center of wind turbines in a rectangular grid for $U_{ref}$ = 8 m/s: (i) top: Horizontal plane and (ii) bottom: Vertical plane at y/D = ±3.5.

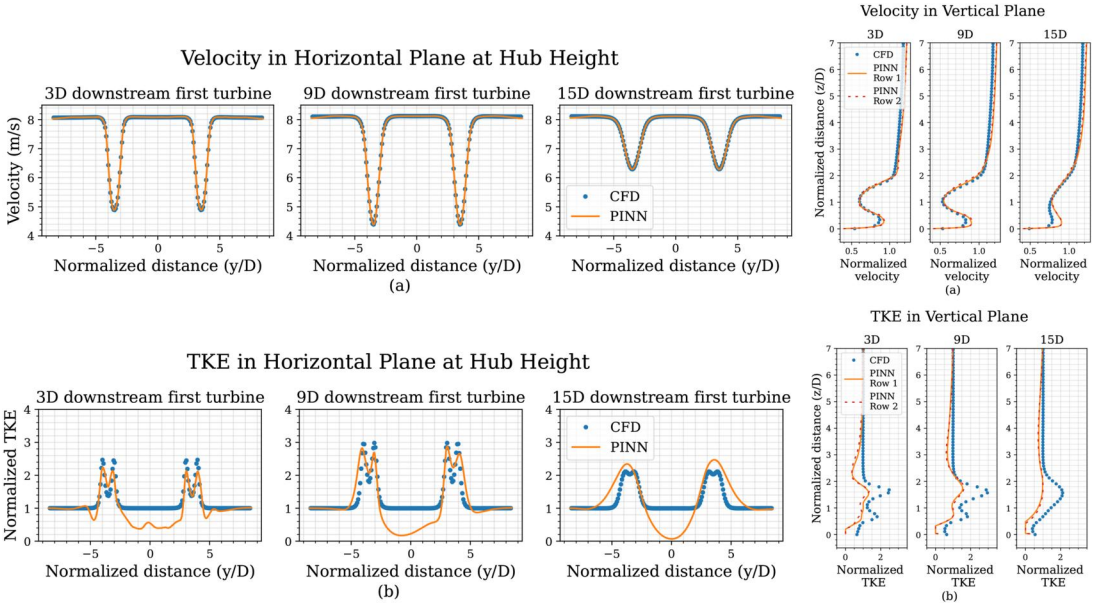



**FIG. 27.** Comparison of (a) velocity and (b) turbulent kinetic energy predicted by PINN model with the numerical solver in two planes at the hub height of the wind turbines in a rectangular grid for $U_{ref}$ = 8 m/s: (i) left: horizontal plane and (ii) right: vertical plane at y/D = ±3.5.

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

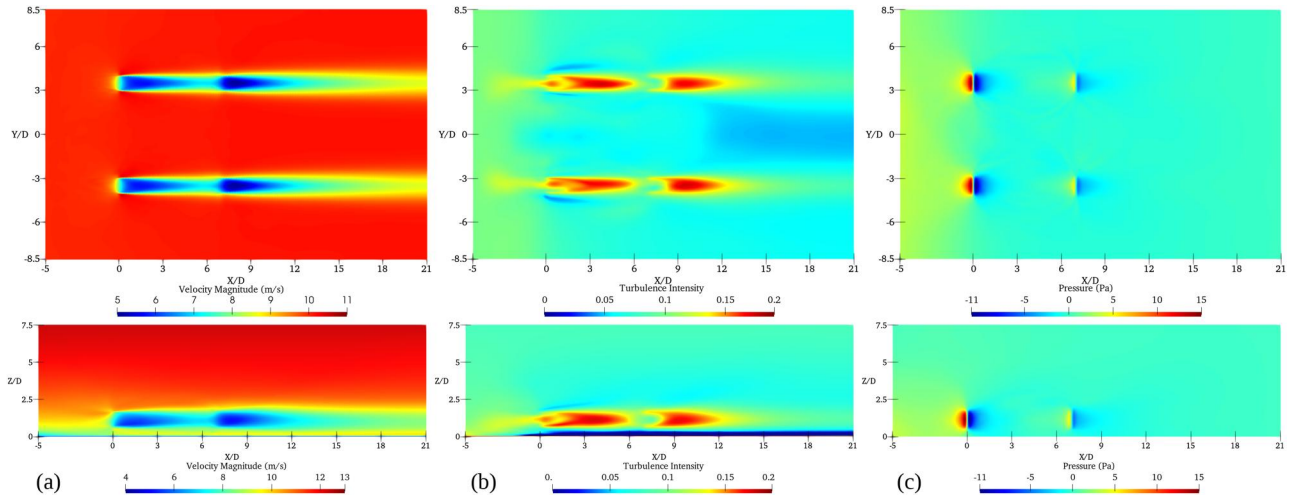



**FIG. 28.** Contours of (a) velocity, (b) turbulence intensity, and (c) pressure in two planes through the center of wind turbines in a rectangular grid for $U_{ref} = 10\,\mathrm{m/s}$: (i) top: horizontal plane and (ii) bottom: vertical plane at y/D = ±3.5.

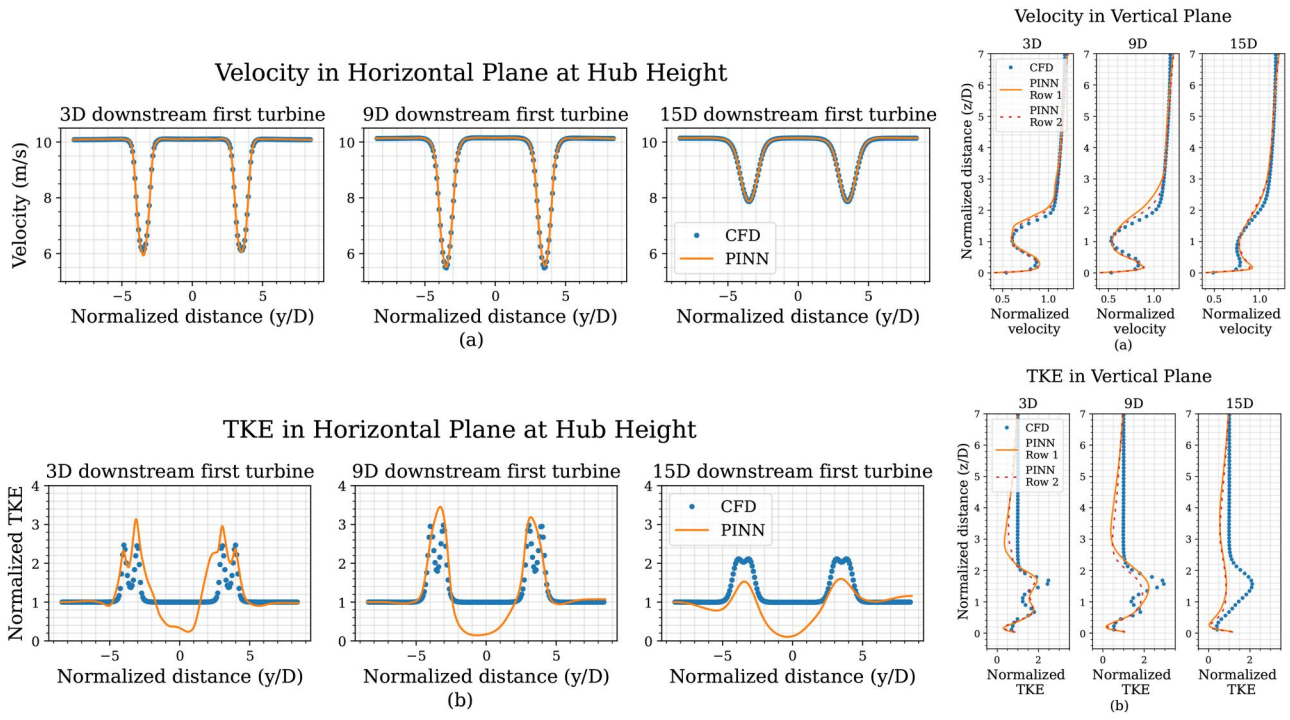



**FIG. 29.** Comparison of (a) velocity and (b) turbulent kinetic energy predicted by PINN model with the numerical solver in two planes at the hub height of the wind turbines in a rectangular grid for $U_{ref} = 10\,\mathrm{m/s}$: (i) left: horizontal plane and (ii) right: vertical plane at y/D = ±3.5.

wind turbines for the rectangular grid. Figure 29 shows the velocity and turbulent kinetic energy profiles in the two planes at different downstream locations, along with the simulation from the CFD k–$\omega$ SST solver for comparison.

### 3. Reference velocity: 11 m/s

Figure 30 shows the contour plot of the RANS-PINN turbulent flow simulations (velocity, turbulence intensity, and pressure) in horizontal and vertical planes, passing through the hub of the

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

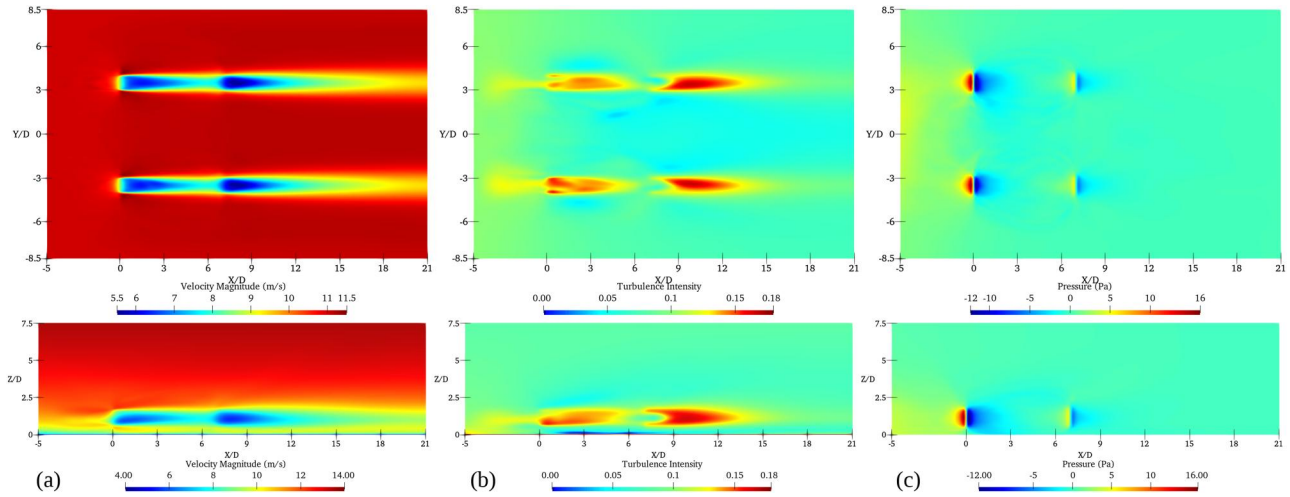



**FIG. 30.** Contours of (a) velocity, (b) turbulence intensity, and (c) pressure in two planes through the center of wind turbines in a rectangular grid for $U_{ref} = 11\,\mathrm{m/s}$: (i) top: Horizontal plane and (ii) bottom: vertical plane at y/D = ±3.5.

wind turbines for the rectangular grid. Figure 31 shows the velocity and turbulent kinetic energy profiles in the two planes at different downstream locations, along with the simulation from the CFD k–$\omega$ SST solver for comparison.

## APPENDIX I: TRANSFER LEARNING: ADDITIONAL PLOTS

Two simulation cases are conducted to demonstrate the effectiveness of transfer learning: (i) how a higher-velocity test case can

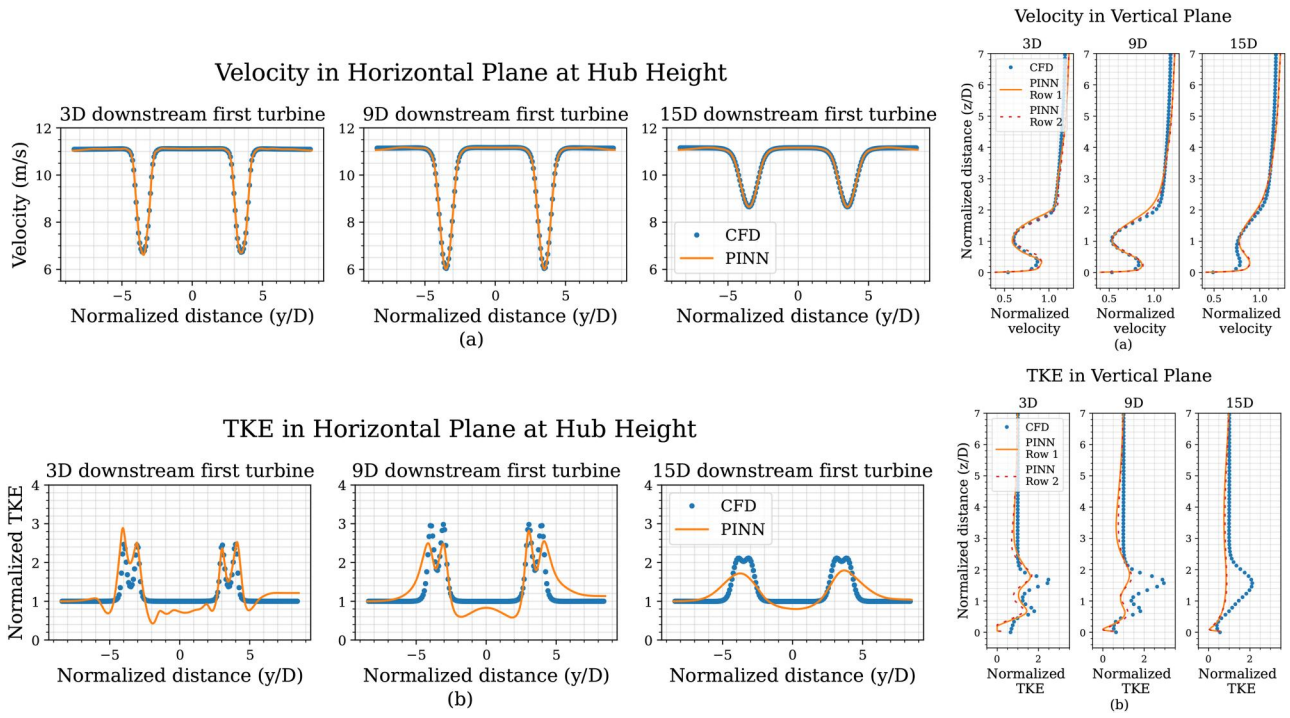



**FIG. 31.** Comparison of (a) velocity and (b) turbulent kinetic energy predicted by PINN model with the numerical solver in two planes at the hub height of the wind turbines in a rectangular grid for $U_{ref} = 11\,\mathrm{m/s}$: (i) left: horizontal plane and (ii) right: vertical plane at y/D = ±3.5.

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

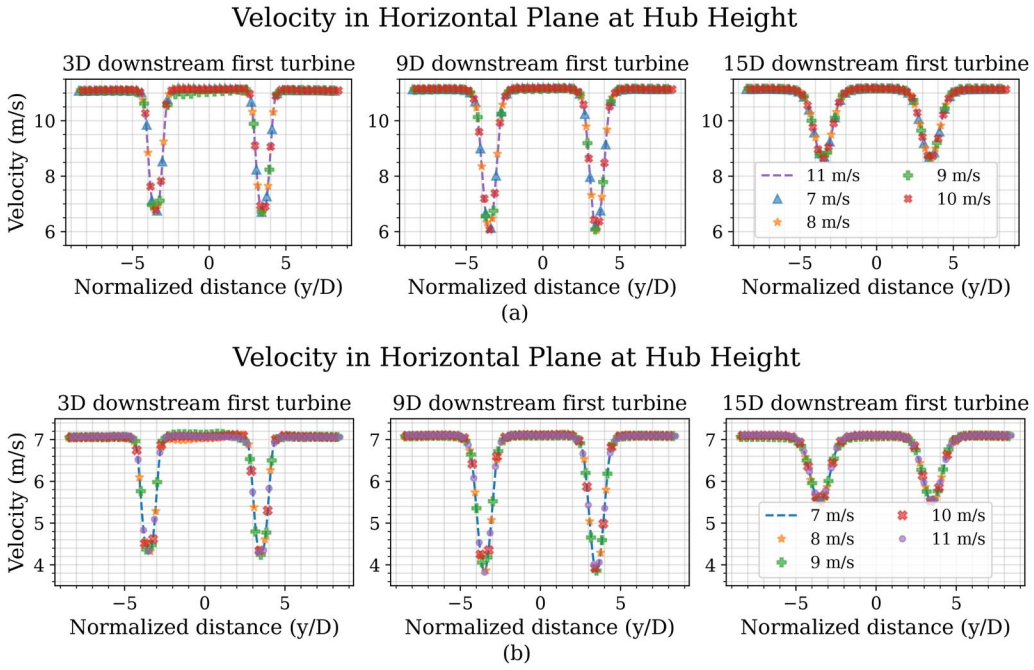



**FIG. 32.** Velocity in a horizontal plane at the hub height of the wind turbines at different downstream locations simulated by the transfer learning strategy. (a) Higher velocity ($U_{ref} = 11$ m/s) simulation from pre-trained lower velocity ($U_{ref} = 7, 8, 9,$ and10 m/s) model and (b) lower velocity ($U_{ref} = 7$ m/s) simulation from pre-trained higher velocity ($U_{ref} = 8, 9, 10,$ and11 m/s) model. The dashed line is the target in each case. Markers denote the simulation after transfer learning, and the marker value is the $U_{ref}$ of the pre-trained model.

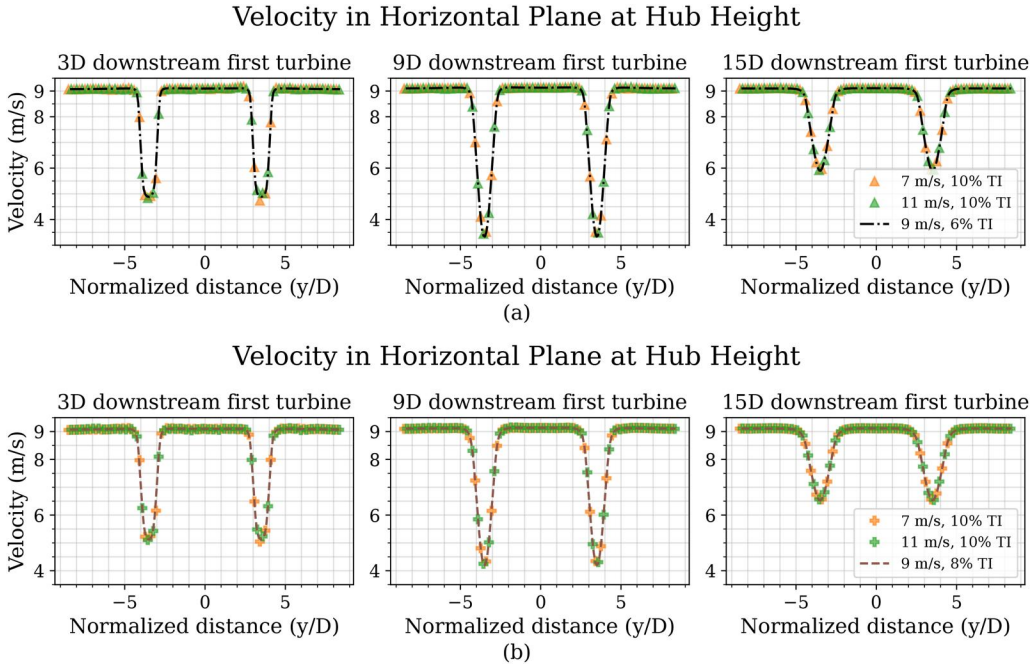



**FIG. 33.** Velocity in a horizontal plane at the hub height of the wind turbines at different downstream locations simulated by the transfer learning strategy. $U = 9$ m/s at turbulence intensities of (a) 6% and (b) 8% simulations from pre-trained model with $U = 7$and11 m/s at turbulence intensity of 10%. The dashed line is the target in each case. Markers denote the simulation after transfer learning, and the marker values indicate velocity and turbulence intensity of the pre-trained model.

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

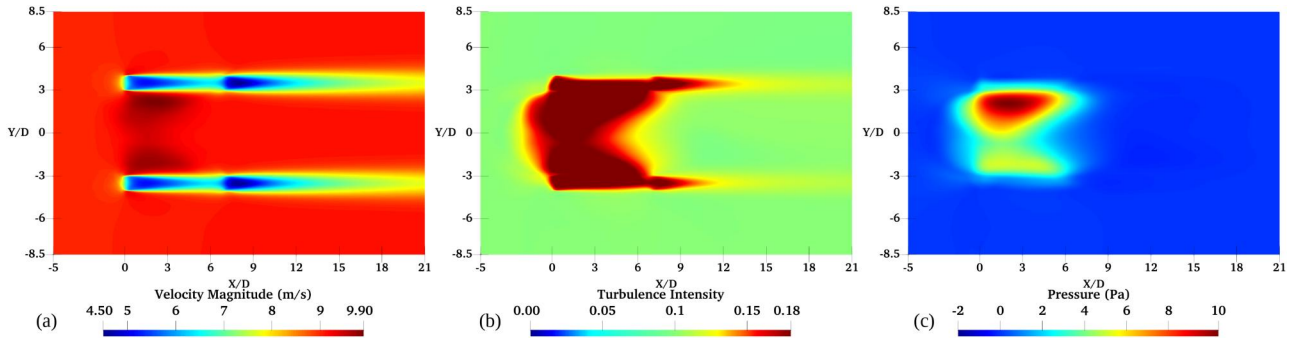



**FIG. 34.** Contours of (a) velocity, (b) turbulence intensity, and (c) pressure in the horizontal plane through the center of wind turbines in a rectangular grid for $U_{ref} = 9\,\mathrm{m/s}$ simulated using a pure data-driven approach.

be quickly simulated from a pre-trained lower-velocity PINN model and vice versa, while maintaining a constant turbulence intensity across the simulations, and (ii) how a test case with different velocity and turbulence intensity can be simulated from a pre-trained model with different conditions. Figures 32 and 33 compare the velocity in a horizontal plane at the hub height of the wind turbines at different downstream locations for case *i* and case *ii*, respectively.

## APPENDIX J: WHY PHYSICS-INFORMED?

Figure 34 illustrates why incorporating physical constraints is essential when only sparse or partial data are available. In this case, velocity measurements were used as the sole supervisory data source. Still, the velocity contours show deviation as indicated by the elevated values between the turbines [Fig. 34(a)]. Because no data were provided for turbulence or pressure fields, purely data-driven learning fails to recover the correct physical behavior in these variables. This is evident in the poor predictions of turbulence intensity [Fig. 34(b)] and pressure [Fig. 34(c)], both of which show clear inconsistencies. These results demonstrate that relying solely on data—especially when limited to a single variable—cannot guarantee physically consistent predictions across all flow quantities. Physics-informed training overcomes this limitation by enforcing the governing equations, enabling the model to infer unmeasured variables in a physically valid manner.

## REFERENCES

[1]P. Moin and K. Mahesh, "Direct numerical simulation: A tool in turbulence research," Annu. Rev. Fluid Mech. **30**, 539–578 (1998).
[2]P. J. Mason, "Large-eddy simulation: A critical review of the technique," Q. J. R. Meteorol. Soc. **120**, 1–26 (1994).
[3]C. G. Speziale, "Analytical methods for the development of reynolds-stress closures in turbulence," Annu. Rev. Fluid Mech. **23**, 107–157 (1991).
[4]J. P. Slotnick, A. Khodadoust, J. Alonso, D. Darmofal, W. Gropp, E. Lurie, and D. J. Mavriplis, "CFD vision 2030 study: A path to revolutionary computational aerosciences," Report No. NF1676L-18332 (NASA, Boeing Research & Technology, Huntington Beach, CA, 2014).
[5]C. D. Pérez-Segarra, O. Lehmkuhl, J. Jaramillo, G. Colomer, and A. Oliva, "Turbulence modelling and numerical issues: From rans to DNS and LES," in *Actas da III Conferência Nacional em Mecânica de Fluidos, Termodinâmica e Energia (MEFTE-BRAGANÇA 09) p3 Paper (in CD) (Actas da III Conferência Nacional em Mecânica de Fluidos, Termodinâmica e Energia (MEFTE-BRAGANÇA 09) p3 Paper (in CD)* [Associação Portuguesa de Mecânica Teórica, Aplicada e Computacional (APMTAC), 2009], pp. 1–31.
[6]M. Breuer, N. Jovičić, and K. Mazaev, "Comparison of DES, RANS and LES for the separated flow around a flat plate at high incidence," Int. J. Numer. Methods Fluids **41**, 357–388 (2003).
[7]X. Zhu, E. Phillips, V. Spandan, J. Donners, G. Ruetsch, J. Romero, R. Ostilla-Mónico, Y. Yang, D. Lohse, R. Verzicco *et al.*, "AFiD-GPU: A versatile navier–stokes solver for wall-bounded turbulent flows on GPU clusters," Comput. Phys. Commun. **229**, 199–210 (2018).
[8]P. Costa, E. Phillips, L. Brandt, and M. Fatica, "GPU acceleration of cans for massively-parallel direct numerical simulations of canonical fluid flows," Comput. Math. Appl. **81**, 502–511 (2021).
[9]M. Bernardini, D. Modesti, F. Salvadore, and S. Pirozzoli, "Streams: A high-fidelity accelerated solver for direct numerical simulation of compressible turbulent flows," Comput. Phys. Commun. **263**, 107906 (2021).
[10]F. De Vanna, F. Avanzi, M. Cogo, S. Sandrin, M. Bettencourt, F. Picano, and E. Benini, "Uranos: A GPU accelerated navier-stokes solver for compressible wall-bounded flows," Comput. Phys. Commun. **287**, 108717 (2023).
[11]M. Crialesi-Esposito, N. Scapin, A. D. Demou, M. E. Rosti, P. Costa, F. Spiga, and L. Brandt, "Flutas: A GPU-accelerated finite difference code for multiphase flows," Comput. Phys. Commun. **284**, 108602 (2023).
[12]A. Radhakrishnan, H. Le Berre, B. Wilfong, J.-S. Spratt, M. Rodriguez, Jr., T. Colonius, and S. H. Bryngelson, "Method for scalable and performant GPU-accelerated simulation of multiphase compressible flow," Comput. Phys. Commun. **302**, 109238 (2024).
[13]A. Di Bella, M. Raissi, D. Santoro, and P. Roccaro, "Physics-informed neural networks in water and wastewater systems: A critical review," Water Res. **293**, 125449 (2026).
[14]J. D. Toscano, V. Oommen, A. J. Varghese, Z. Zou, N. Ahmadi Daryakenari, C. Wu, and G. E. Karniadakis, "From pinns to pikans: Recent advances in physics-informed machine learning," Mach. Learn. Comput. Sci. Eng. **1**, 1–43 (2025).
[15]L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via deeponet based on the universal approximation theorem of operators," Nat. Mach. Intell. **3**, 218–229 (2021).
[16]Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," arXiv:2010.08895 (2020).
[17]R. Alaifari, F. Bartolucci, E. de Bezenac, R. Molinaro, S. Mishra, and B. Raonic, "Representation equivalent neural operators: A framework for alias-free operator learning," Adv. Neural Inf. Process. Syst. **36**, 69661–69672 (2024).
[18]N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Neural operator: Learning maps between function spaces with applications to PDEs," J. Mach. Learn. Res. **24**, 1–97 (2023).
[19]M. Optis and J. Perr-Sauer, "The importance of atmospheric turbulence and stability in machine-learning models of wind farm power production," Renewable Sustainable Energy Rev. **112**, 27–41 (2019).

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

[20]Z. Ti, X. W. Deng, and H. Yang, "Wake modeling of wind turbines using machine learning," Appl. Energy **257**, 114025 (2020).
[21]Z. Ti, X. W. Deng, and M. Zhang, "Artificial neural networks based wake model for power prediction of wind farm," Renewable Energy **172**, 618–631 (2021).
[22]J. Zhang and X. Zhao, "A novel dynamic wind farm wake model based on deep learning," Appl. Energy **277**, 115552 (2020).
[23]Z. Zhang, C. Santoni, T. Herges, F. Sotiropoulos, and A. Khosronejad, "Time-averaged wind turbine wake flow field prediction using autoencoder convolutional neural networks," Energies **15**, 41 (2021).
[24]P. M. Doran, "Chapter 7 - fluid flow," in *Bioprocess Engineering Principles*, 2nd ed., edited by P. M. Doran (Academic Press, London, 2013), pp. 201–254.
[25]M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," J. Comput. Phys. **378**, 686–707 (2019).
[26]A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: A survey," J. Mach. Learn. Res. **18**, 1–43 (2018).
[27]A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, and M. W. Mahoney, "Characterizing possible failure modes in physics-informed neural networks," Adv. Neural Inf. Process. Syst. **34**, 26548–26560 (2021).
[28]S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," SIAM J. Sci. Comput. **43**, A3055–A3081 (2021).
[29]Y. Sun, U. Sengupta, and M. Juniper, "Physics-informed deep learning for simultaneous surrogate modeling and pde-constrained optimization of an airfoil geometry," Comput. Methods Appl. Mech. Eng. **411**, 116042 (2023).
[30]X. Jin, S. Cai, H. Li, and G. E. Karniadakis, "NSFnets (Navier–Stokes flow nets): Physics-informed neural networks for the incompressible navier-stokes equations," J. Comput. Phys. **426**, 109951 (2021).
[31]S. Luo, M. Vellakal, S. Koric, V. Kindratenko, and J. Cui, "Parameter identification of rans turbulence model using physics-embedded neural network," in *International Conference on High Performance Computing* (Springer, 2020), pp. 137–149.
[32]O. Hennigh, S. Narasimhan, M. A. Nabian, A. Subramaniam, K. Tangsali, Z. Fang, M. Rietmann, W. Byeon, and S. Choudhry, "Nvidia simnet$^{TM}$: An ai-accelerated multi-physics simulation framework," in *International Conference on Computational Science* (Springer, 2021), pp. 447–461.
[33]S. Ghosh, A. Chakraborty, G. O. Brikis, and B. Dey, "RANS-PINN based simulation surrogates for predicting turbulent flows," arXiv:2306.06034 (2023).
[34]H. Eivazi, M. Tahani, P. Schlatter, and R. Vinuesa, "Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations," Phys. Fluids **34**, 075117 (2022).
[35]S. K. Biswas and N. Anand, "Three-dimensional laminar flow using physics informed deep neural networks," Phys. Fluids **35**, 121703 (2023).
[36]S. Hu, M. Liu, S. Zhang, S. Dong, and R. Zheng, "Physics-informed neural network combined with characteristic-based split for solving Navier–Stokes equations," Eng. Appl. Artif. Intell. **128**, 107453 (2024).
[37]E.-Z. Rui, Z.-W. Chen, Y.-Q. Ni, L. Yuan, and G.-Z. Zeng, "Reconstruction of 3D flow field around a building model in wind tunnel: A novel physics-informed neural network framework adopting dynamic prioritization self-adaptive loss balance strategy," Eng. Appl. Comput. Fluid Mech. **17**, 2238849 (2023).
[38]P. Clark Di Leoni, K. Agarwal, T. A. Zaki, C. Meneveau, and J. Katz, "Reconstructing turbulent velocity and pressure fields from under-resolved noisy particle tracks using physics-informed neural networks," Exp. Fluids **64**, 95 (2023).
[39]G. S. Hora, P. Gentine, M. Momen, and M. G. Giometto, "Physics-informed data-driven reconstruction of turbulent wall-bounded flows from planar measurements," Phys. Fluids **36**, 115191 (2024).
[40]J. Zhang and X. Zhao, "Spatiotemporal wind field prediction based on physics-informed deep learning and lidar measurements," Appl. Energy **288**, 116641 (2021).
[41]J. Zhang and X. Zhao, "Three-dimensional spatiotemporal wind field reconstruction based on physics-informed deep learning," Appl. Energy **300**, 117390 (2021).
[42]A. Gafoor CTP, S. K. Boya, R. Jinka, A. Gupta, A. Tyagi, S. Sarkar, and D. N. Subramani, "A physics-informed neural network for turbulent wake simulations behind wind turbines," Phys. Fluids **37**, 015110 (2025).
[43]P. Cobelli, K. Shukla, S. Nesmachnow, and M. Draper, "Physics informed neural networks for wind field modeling in wind farms," J. Phys.: Conf. Ser. **2505**, 012051 (2023).
[44]J. Zhang and X. Zhao, "Digital twin of wind farms via physics-informed deep learning," Energy Convers. Manage. **293**, 117507 (2023).
[45]L. Wang, M. Chen, Z. Luo, B. Zhang, J. Xu, Z. Wang, and A. C. Tan, "Dynamic wake field reconstruction of wind turbine through physics-informed neural network and sparse LiDAR data," Energy **291**, 130401 (2024).
[46]J. Song, L. Wang, Z. Xin, and H. Wang, "Wake field prediction of a wind farm based on a physics-informed neural network with different spatiotemporal prediction performance improvement strategies," Theor. Appl. Mech. Lett. **15**, 100577 (2025).
[47]K. Liu, K. Luo, Y. Cheng, A. Liu, H. Li, J. Fan, and S. Balachandar, "Parameterized physics-informed neural networks (p-pinns) solution of uniform flow over an arbitrarily spinning spherical particle," Int. J. Multiphase Flow **180**, 104937 (2024).
[48]W. Cao, S. Tang, Q. Ma, W. Ouyang, and W. Zhang, "Solving all laminar flows around airfoils all-at-once using a parametric neural network solver," arXiv:2501.01165 (2025).
[49]Z. Cao, K. Liu, K. Luo, Y. Cheng, and J. Fan, "Efficient optimization design of flue deflectors through parametric surrogate modeling with physics-informed neural networks," Phys. Fluids **35**, 125149 (2023).
[50]V. Kag, K. Seshasayanan, and V. Gopinath, "Physics-informed data based neural networks for two-dimensional turbulence," Phys. Fluids **34**, 055130 (2022).
[51]S. Wang, S. Sankaran, X. Fan, P. Stinis, and P. Perdikaris, "Simulating three-dimensional turbulence with physics-informed neural networks," arXiv:2507.08972 (2025).
[52]S. Cai, Z. Mao, Z. Wang, M. Yin, and G. E. Karniadakis, "Physics-informed neural networks (pinns) for fluid mechanics: A review," Acta Mech. Sin. **37**, 1727–1738 (2021).
[53]S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, "Scientific machine learning through physics–informed neural networks: Where we are and what's next," J. Sci. Comput. **92**, 88 (2022).
[54]C. Zhao, F. Zhang, W. Lou, X. Wang, and J. Yang, "A comprehensive review of advances in physics-informed neural networks and their applications in complex fluid dynamics," Phys. Fluids **36**, 101301 (2024).
[55]B. E. Launder and B. I. Sharma, "Application of the energy-dissipation model of turbulence to the calculation of flow near a spinning disc," Lett. Heat Mass Transfer **1**, 131–137 (1974).
[56]A. Crespo, F. Manuel, D. Moreno, E. Fraga, and J. Hernandez, "Numerical analysis of wind turbine wakes," in *Workshop on Wind Energy Applications, Delphi, Greece* (Workshop on Wind Energy Applications, Delphi, Greece, 1985), pp. 15–25.
[57]P. Rathore, W. Lei, Z. Frangella, L. Lu, and M. Udell, "Challenges in training pinns: A loss landscape perspective," arXiv:2402.01868 (2024).
[58]W. T. Leung, G. Lin, and Z. Zhang, "NH-PINN: Neural homogenization-based physics-informed neural network for multiscale problems," J. Comput. Phys. **470**, 111539 (2022).
[59]R. Bischof and M. A. Kraus, "Multi-objective loss balancing for physics-informed deep learning," Comput. Methods Appl. Mech. Eng. **439**, 117914 (2025).
[60]B. E. Launder and D. B. Spalding, "The numerical computation of turbulent flows," Comput. Methods Appl. Mech. Eng. **3**, 269–289 (1974).
[61]G. Kalitzin, G. Medic, G. Iaccarino, and P. Durbin, "Near-wall behavior of rans turbulence models and implications for wall functions," J. Comput. Phys. **204**, 265–291 (2005).
[62]B. Blocken, T. Stathopoulos, and J. Carmeliet, "CFD simulation of the atmospheric boundary layer: Wall function problems," Atmos. Environ. **41**, 238–252 (2007).
[63]J. N. Sørensen, "Aerodynamic aspects of wind energy conversion," Annu. Rev. Fluid Mech. **43**, 427–448 (2011).
[64]Y.-S. Chen and S.-W. Kim, "Computation of turbulent flow using an extended turbulence closure model," Report No. NAS 1.26: 179204 (NASA, 1987).
[65]A. El Kasmi and C. Masson, "An extended k–ε model for turbulent flow through horizontal-axis wind turbines," J. Wind Eng. Ind. Aerodyn. **96**, 103–122 (2008).

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing

[66]M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, see tensorflow.org for "TensorFlow: Large-scale machine learning on heterogeneous systems" (2015).
[67]A. Gafoor CTP, see https://github.com/quest-lab-iisc/PINN_WakeTurbulence Model "A PINN for turbulent wake simulations" (2024).
[68]D. V. Dung, N. D. Song, P. S. Palar, and L. R. Zuhal, "On the choice of activation functions in physics-informed neural network for solving incompressible fluid flows," AIAA Paper No. 2023-1803, 2023.
[69]D. P. Kingma, "Adam: A method for stochastic optimization," arXiv:1412.6980 (2014).
[70]X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics* (JMLR, 2010), pp. 249–256.
[71]A. Gafoor CTP (2026). "A PINN for multi-turbine turbulent wake simulation using physics-informed neural networks," GitHub and Zenodo. https://github.com/quest-lab-iisc/PINNs_multi_turbine_wake_simulation
[72]R. E. Froude, "On the part played in propulsion by differences of fluid pressure," Trans. Inst. Nav. Archit. **30**, 390 (1889).
[73]J. N. Sørensen and A. Myken, "Unsteady actuator disc model for horizontal axis wind turbines," J. Wind Eng. Ind. Aerodyn. **39**, 139–149 (1992).
[74]P. J. Richards and R. P. Hoxey, "Appropriate boundary conditions for computational wind engineering models using the k-$\varepsilon$ turbulence model," J. Wind Eng. Ind. Aerodyn. **46–47**, 145–153 (1993).
[75]A. Parente, C. Gorlé, J. Van Beeck, and C. Benocci, "Improved k–$\varepsilon$ model and wall function formulation for the rans simulation of abl flows," J. Wind Eng. Ind. Aerodyn. **99**, 267–278 (2011).
[76]J. Hammersley and D. Handscomb, *Monte Carlo Methods* (Methuen & Co, Ltd., London, 1964), Vol. 40, p. 32.
[77]C. Wu, M. Zhu, Q. Tan, Y. Kartha, and L. Lu, "A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks," Comput. Methods Appl. Mech. Eng. **403**, 115671 (2023).
[78]L. Vermeer, J. N. Sørensen, and A. Crespo, "Wind turbine wake aerodynamics," Prog. Aerosp. Sci. **39**, 467–510 (2003).
[79]Y.-T. Wu and F. Porté-Agel, "Atmospheric turbulence effects on wind-turbine wakes: An Les study," Energies **5**, 5340–5362 (2012).
[80]P. K. Jha, E. P. Duque, J. L. Bashioum, and S. Schmitz, "Unraveling the mysteries of turbulence transport in a wind farm," Energies **8**, 6468–6496 (2015).
[81]M. Puccioni, C. Moss, C. Jacquet, and G. Iungo, "Blockage and speedup in the proximity of an onshore wind farm: A scanning wind lidar experiment," J. Renewable Sustainable Energy **15**, 053307 (2023).
[82]H. Lu, X. Gao, Z. Xu, H. Xiao, Y. Gao, H. Zhang, H. Ma, Y. Zhu, X. Zhu, and Y. Wang, "Wind direction prediction combined with wind speed in a wind farm," Energy **333**, 137334 (2025).
[83]T. Mikkelsen, N. Angelou, K. Hansen, M. Sjöholm, M. Harris, C. Slinger, P. Hadley, R. Scullion, G. Ellis, and G. Vives, "A spinner-integrated wind lidar for enhanced wind turbine control," Wind Energy **16**, 625–643 (2013).
[84]N. Angelou and M. Sjöholm, "Data reliability enhancement for wind-turbine-mounted lidars," Remote Sens. **14**, 3225 (2022).
[85]M. F. Van Dooren, D. Trabucchi, and M. Kühn, "A methodology for the reconstruction of 2d horizontal wind fields of wind turbine wakes based on dual-doppler lidar measurements," Remote Sens. **8**, 809 (2016).
[86]J. P. Goit, S. Shimada, and T. Kogaki, "Can lidars replace meteorological masts in wind energy?," Energies **12**, 3680 (2019).
[87]G. V. Iungo, "Experimental characterization of wind turbine wakes: Wind tunnel tests and wind lidar measurements," J. Wind Eng. Ind. Aerodyn. **149**, 35–39 (2016).
[88]Siemens Digital Industries Software, see https://www.plm.automation.siemens.com/global/en/products/simcenter/STAR-CCM.html for "Simcenter STAR-CCM+, version 2021.1" (Siemens 2021).
[89]F. R. Menter, "Two-equation eddy-viscosity turbulence models for engineering applications," AIAA J. **32**, 1598–1605 (1994).
[90]E. G. Antonini, D. A. Romero, and C. H. Amon, "Analysis and modifications of turbulence models for wind turbine wake simulations in atmospheric boundary layers," J. Sol. Energy Eng. **140**, 031007 (2018).
[91]A. Rezaeiha, H. Montazeri, and B. Blocken, "On the accuracy of turbulence models for CFD simulations of vertical axis wind turbines," Energy **180**, 838–857 (2019).
[92]N. Stergiannis, J. van Beeck, and M. C. Runacres, "Full HAWT rotor CFD simulations using different rans turbulence models compared with actuator disk and experimental measurements," Wind Energy Sci. Discuss. **2017**, 1–20.
[93]S. K. Boya and D. N. Subramani, "PINTO: Physics-informed transformer neural operator for learning generalized solutions of partial differential equations for any initial and boundary condition," Comput. Phys. Commun. **315**, 109702 (2025).
[94]S. Goswami, C. Anitescu, S. Chakraborty, and T. Rabczuk, "Transfer learning enhanced physics informed neural network for phase-field modeling of fracture," Theor. Appl. Fract. Mech. **106**, 102447 (2020).
[95]S. Desai, M. Mattheakis, H. Joy, P. Protopapas, and S. Roberts, "One-shot transfer learning of physics-informed neural networks," arXiv:2110.11286 (2021).
[96]R. D. Moser, J. Kim, and N. N. Mansour, "Direct numerical simulation of turbulent channel flow up to Re $\tau = 590$," Phys. Fluids **11**, 943–945 (1999).

12 September 2026 13:03:32


Published under an exclusive license by AIP Publishing